

学习内容

CONTENTS



1

神经元与神经网络

2

BP神经网络

3

卷积神经网络

4

生成对抗网络

5

循环神经网络

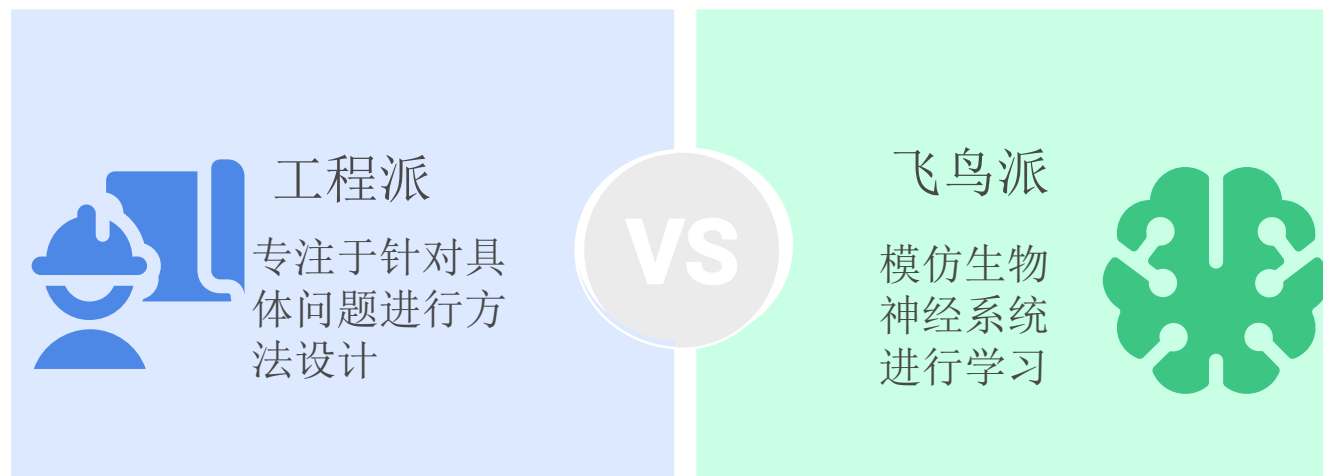
6

Transformer

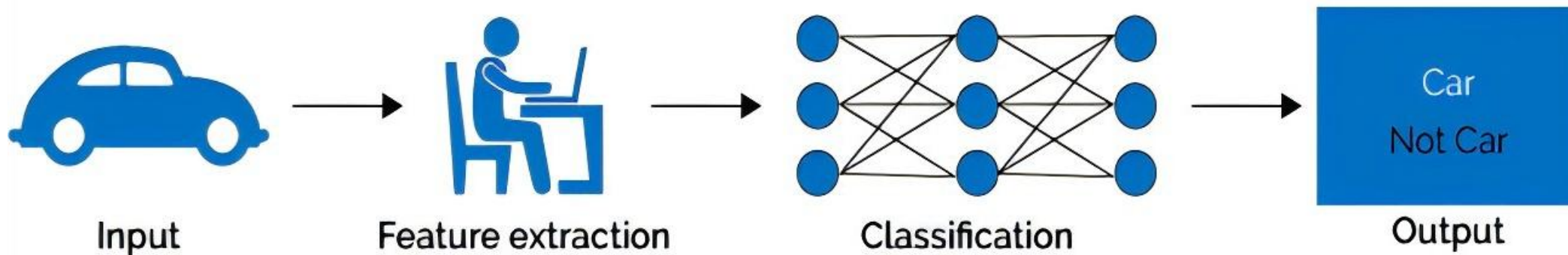
传统机器学习存在已久，为什么
近年来引起广泛关注？



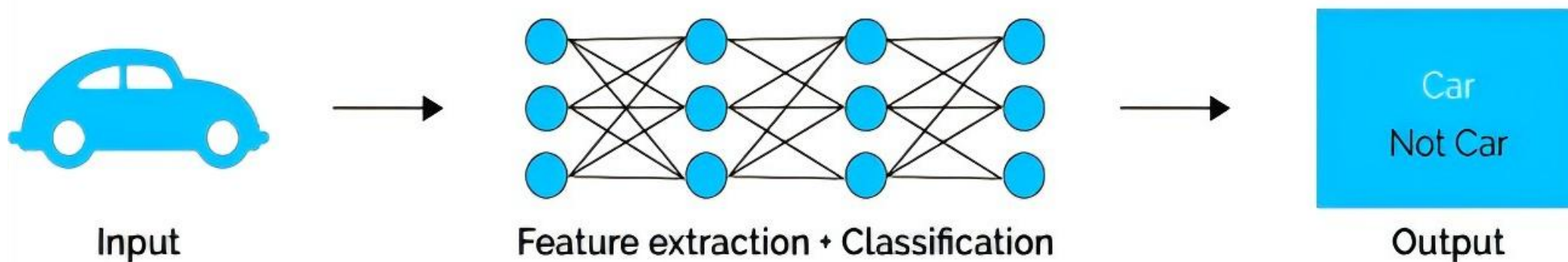
深度神经网络为代表的人工智能
取得了重大突破。
深度学习是以复杂神经网络为模
型的机器学习。



Machine Learning

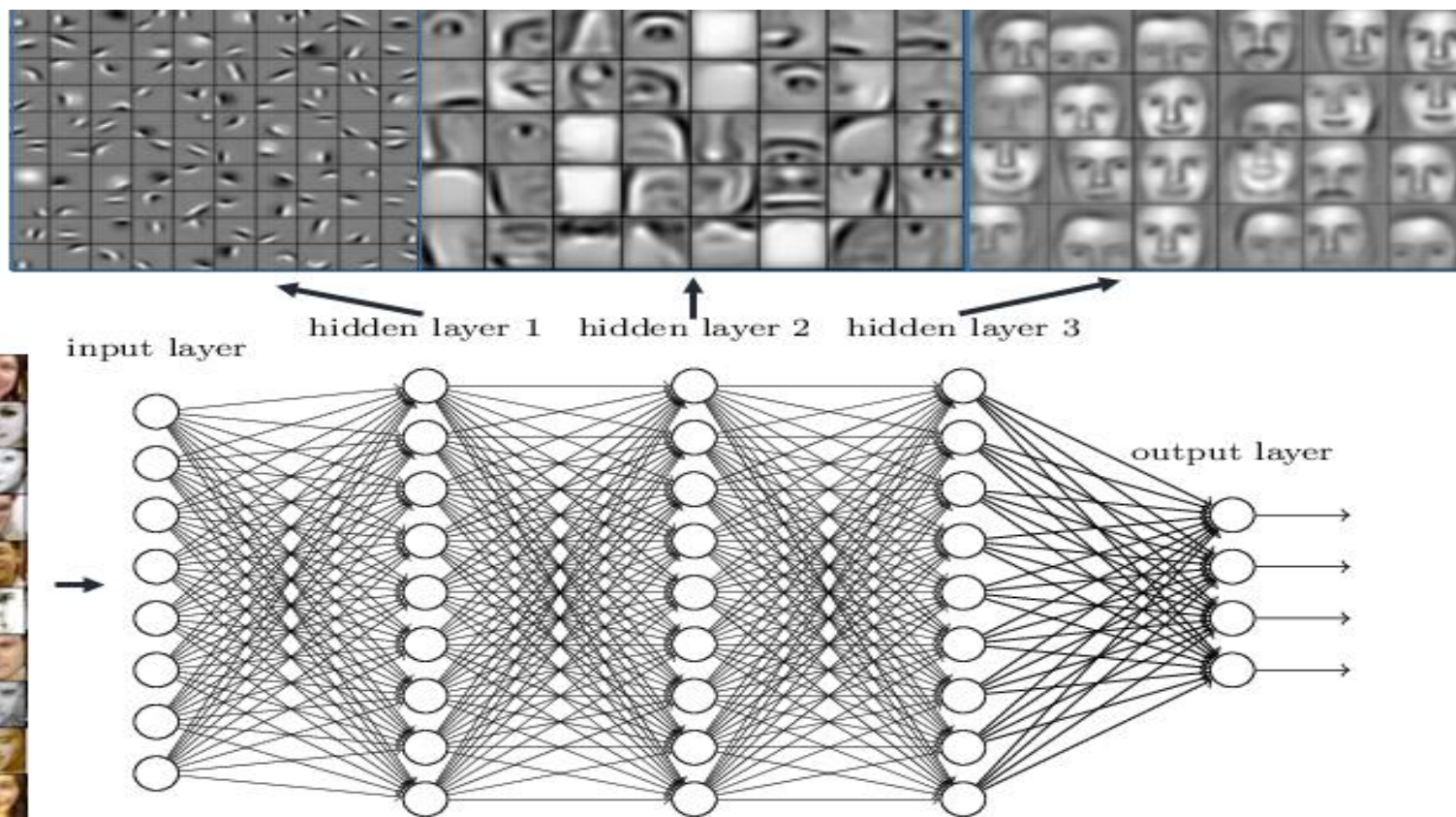


Deep Learning

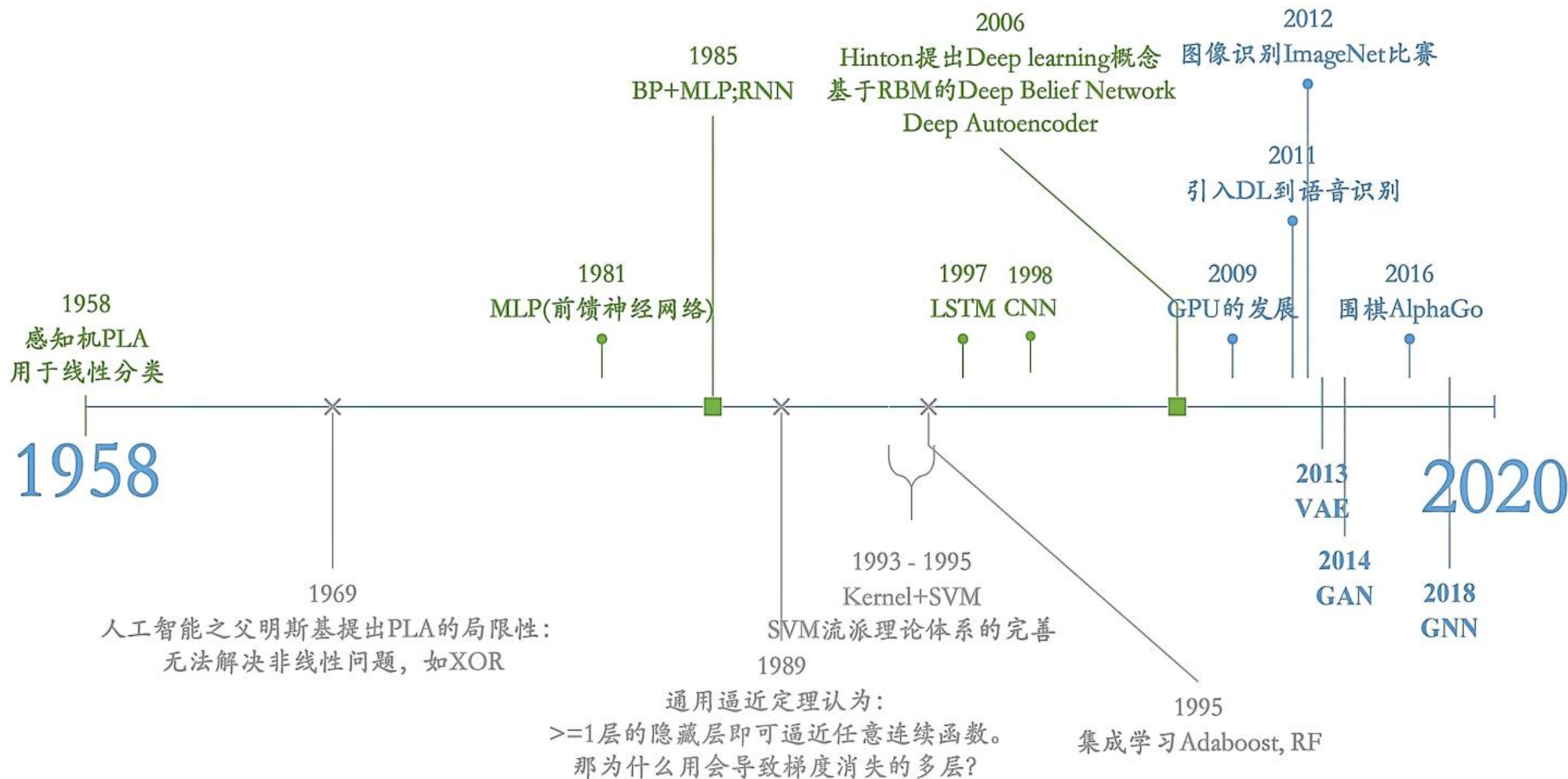


理论研究表明，**深层神经网络具备比浅层神经网络超强的函数表达能力**
(*Bengio et al., 2006a; Bengio and LeCun, 2007, Delalleau 2011*)

Deep neural
networks learn
hierarchical feature
representations



突出**特征学习**的重要性;
(逐层抽象) 训练机制;



主流深度学习框架

百度PaddlePaddle(2016.9)
华为MindSpore(2018.10)

库名	发布者	支持语言	支持系统
TensorFlow	Google	Python/C++/ Java/Go	Linux/Mac OS/ Windows/Android/iOS
Caffe	UC Berkeley	Python/C++/ Matlab	Linux/Mac OS/Windows
CNTK	Microsoft	Python/C++/ BrainScript	Linux/Windows
MXNet	DMLC (分布式机器学习社区)	Python/C++/Matlab/ Julia/Go/R/Scala	Linux/Mac OS/ Windows/Android/iOS
Torch/Pytorch	Facebook	C/Lua/Python	Linux/Mac OS/ Windows/Android/iOS
Theano	蒙特利尔大学	Python	Linux/Mac OS/Windows
Neon	Intel	Python	Linux

深度学习模型部署推理框架

<https://baijiahao.baidu.com/s?id=1714111587892605090&wfr=spider&for=pc>



深度学习模型部署推理框架: *ONNX*、*NCNN*、*OpenVINO*、*TensorRT*

深度学习模型部署推理框架

<https://zhuanlan.zhihu.com/p/667682419>

模型推理部署框架	应用平台
NCNN	移动端
OpenVINO	CPU，GPU，嵌入式平台都可以使用，尤其是在CPU上首选OPenVINO。DepthAI嵌入式空间AI平台。
TensorRT	只能用在NIVDIA的GPU上的推理框架。NIVDIA的Jetson平台。
Mediapipe	服务端，移动端，嵌入式平台，TPU。

- 腾讯公司开发的移动端平台部署工具——NCNN;
- Intel公司针对自家设备开开发的部署工具——OpenVINO;
- NVIDIA公司针对自家GPU开发的部署工具——TensorRT;
- Google针对自家硬件设备和深度学习框架开发的部署工具——Mediapipe;
- 由微软、亚马逊、Facebook 和 IBM 等公司共同开发的开放神经网络交换格式——ONNX;

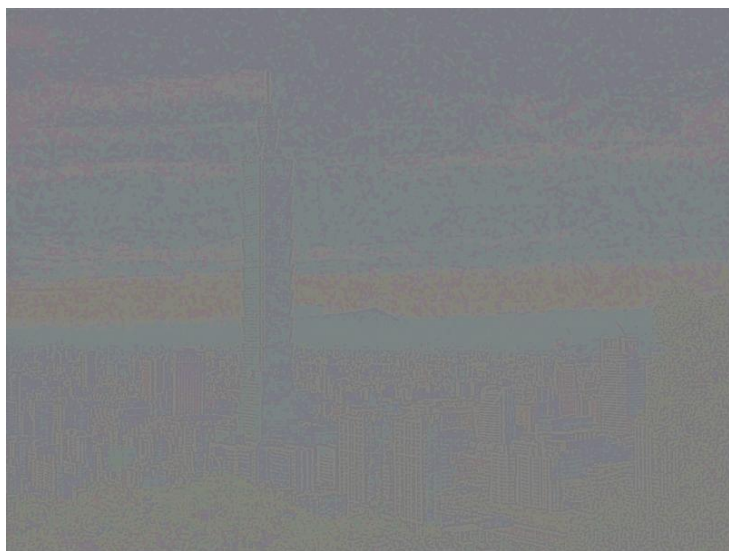
深度学习模型部署推理框架: *ONNX*、*NCNN*、*OpenVINO*、*TensorRT*

内容

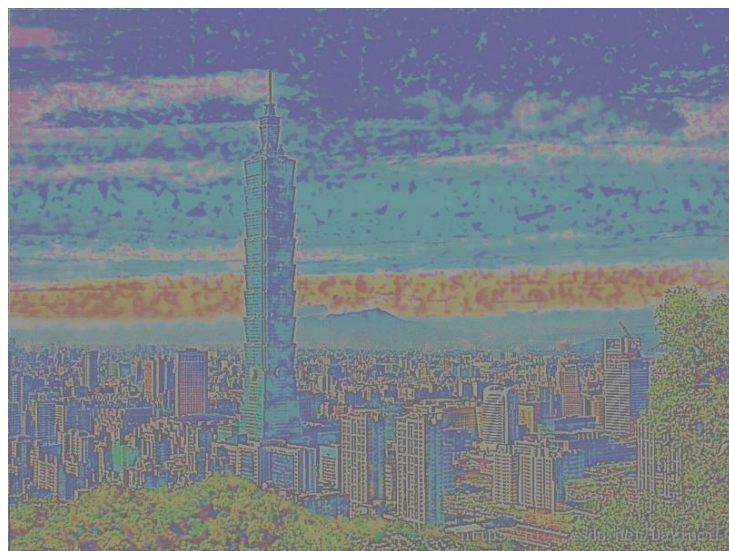


风格

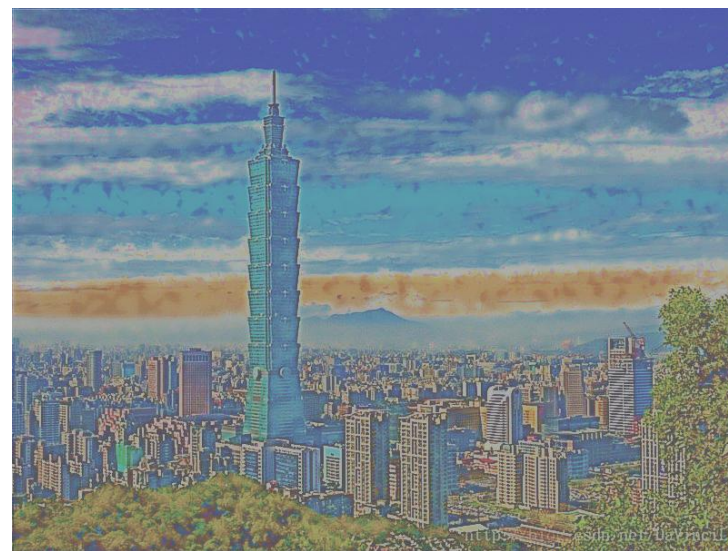




迭代50次



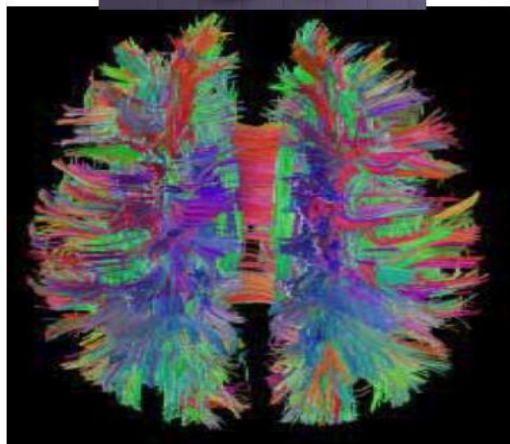
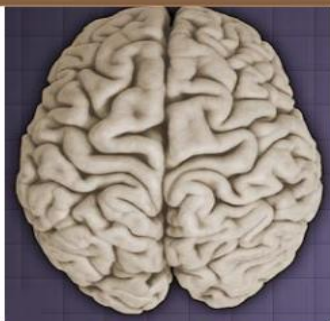
迭代100次



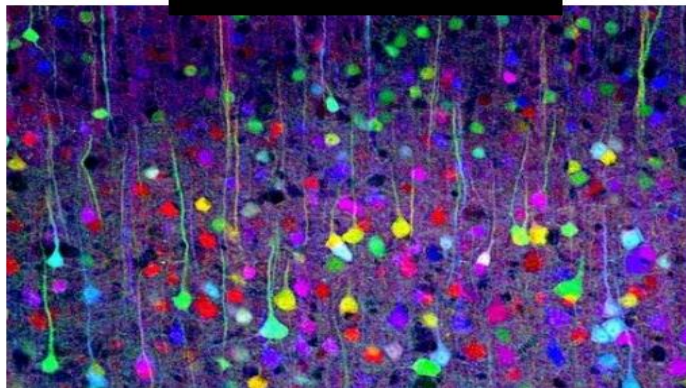
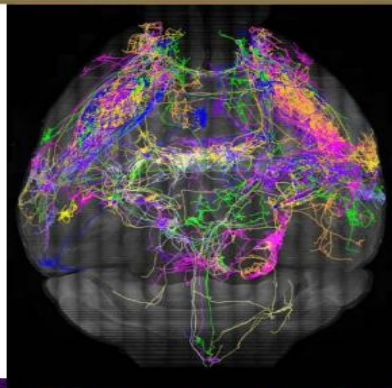
迭代200次

人脑由近千亿 (10^{11}) 神经细胞 (**神经元**) 通过百万亿 (10^{14}) 个突触联结组成**神经网络**，以特殊神经环路实现感知、运动、思维等功能。

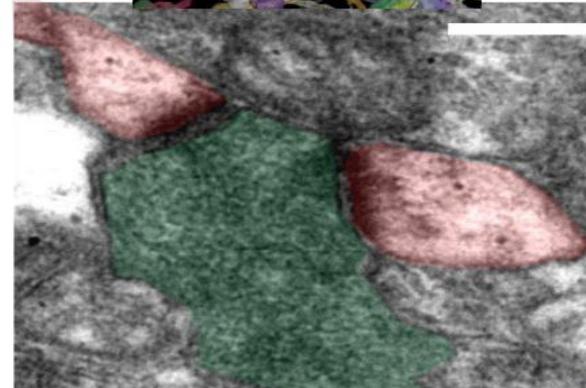
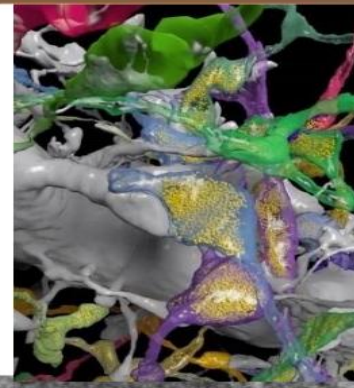
宏观图谱 (macroscopic)
核磁共振脑成像 (MRI)
空间分辨率: 毫米



介观图谱 (mesoscopic)
光学显微镜成像
空间分辨率: 微米



微观图谱 (microscopic)
电子显微镜成像
空间分辨率: 纳米

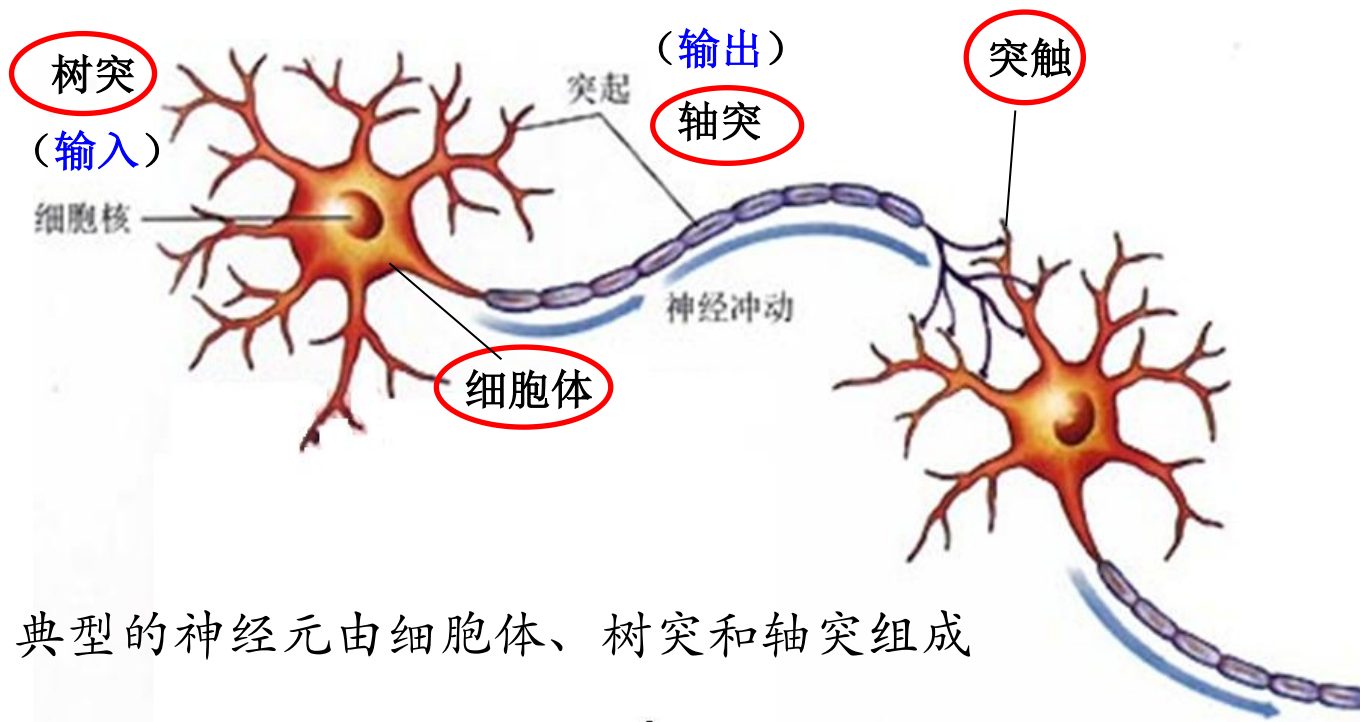




那就叫小苦吧 bilibili

神经元

生物神经元

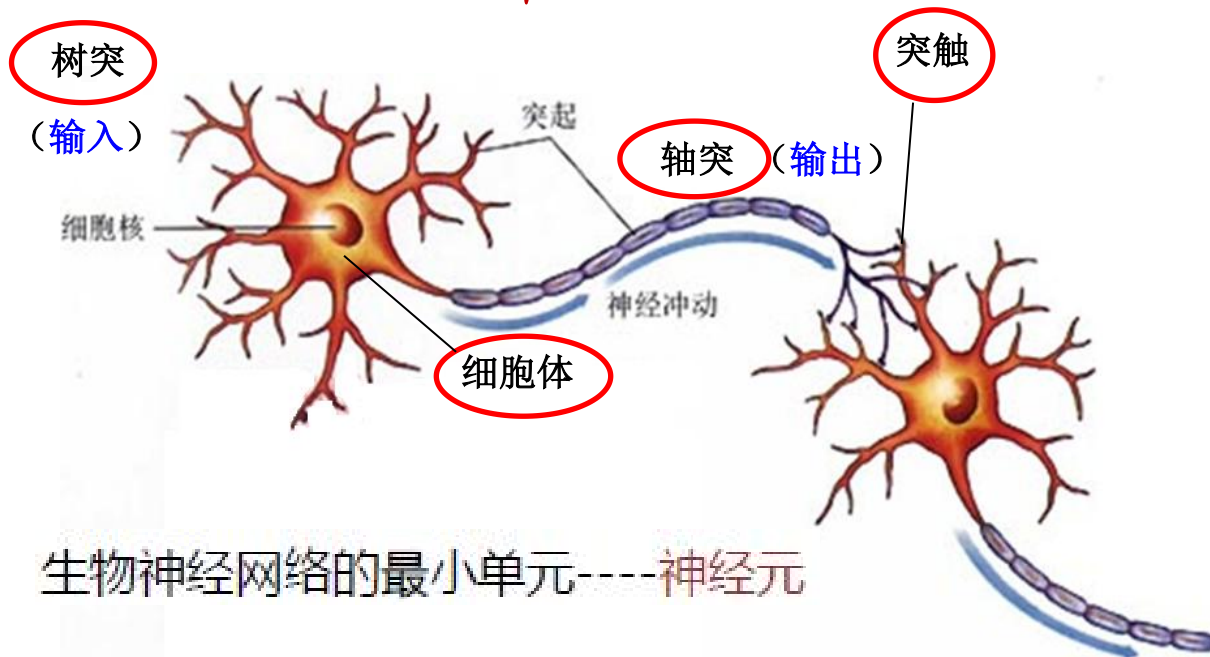


典型的神经元由细胞体、树突和轴突组成

神经元结构的工作机制

- 神经元状态：兴奋/抑制
- 输入兴奋电位总量超过某个**阈值**，神经元会被激发进入**兴奋状态**，发出输出脉冲，并由突触传递给其他神经元。
- 学习与遗忘：由于神经元结构的可塑性，突触的传递作用可增强和减弱。

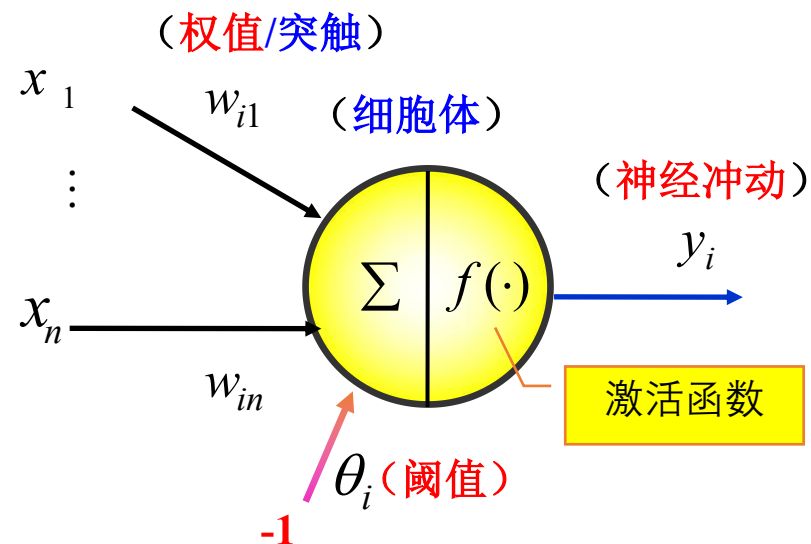
生物神经元



1943 年，心理学家 **Warren McCulloch** 和逻辑学家 **Walter Pitts** 在《A logical calculus of the ideas immanent in nervous activity》中，根据生物神经元(Neuron)结构，提出了最早的人工神经元数学模型，称为 **MP 神经元模型**。



人工神经元

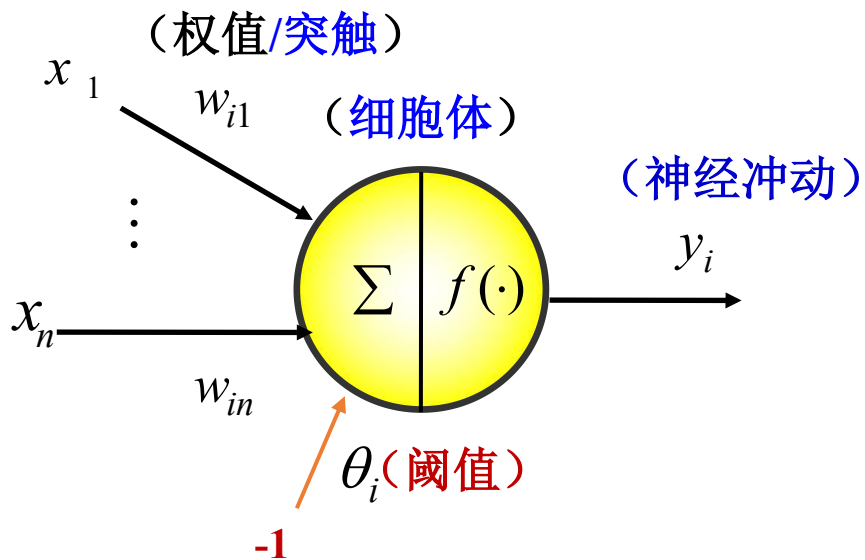


$$y_i = f(w_{i1}x_1 + w_{i2}x_2 + \dots + w_{in}x_n - \theta_i)$$

$$= f\left(\sum_{j=1}^n w_{ij}x_j - \theta_i\right) = f\left(\sum_{j=0}^n w_{ij}x_j\right)$$

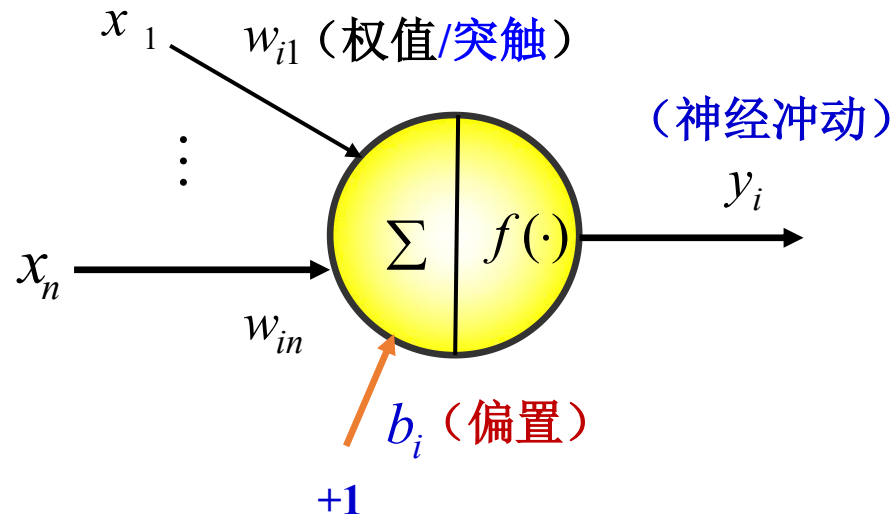
$$= f\left[\begin{bmatrix} w_{i0} & w_{i1} & \dots & w_{in} \end{bmatrix} \cdot \begin{bmatrix} -1 \\ x_1 \\ \vdots \\ x_n \end{bmatrix}\right] = f(\mathbf{w}_i^T \mathbf{x})$$

$(x_0 = -1, w_{i0} = \theta_i)$

人工神经元 i 的模型图

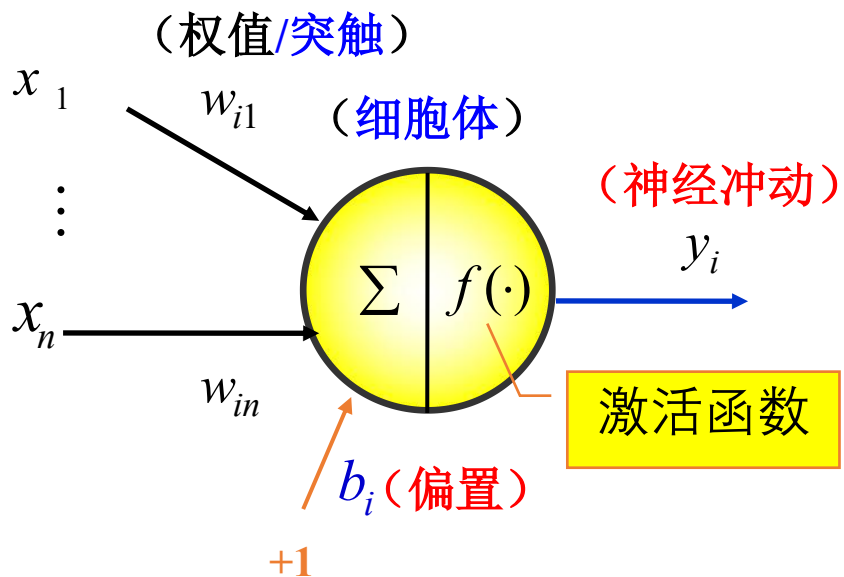
$$y_i = f\left(\sum_{j=1}^n w_{ij}x_j - \theta_i\right) = f\left(\sum_{j=0}^n w_{ij}x_j\right) = f(\mathbf{w}^T \mathbf{x})$$

$(x_0 = -1, w_{i0} = \theta_i)$

人工神经元 i 的模型图

$$y_i = f\left(\sum_{j=1}^n w_{ij}x_j + b_i\right) = f\left(\sum_{j=0}^n w_{ij}x_j\right) = f(\mathbf{w}^T \mathbf{x})$$

$(x_0 = 1, w_{i0} = b_i)$

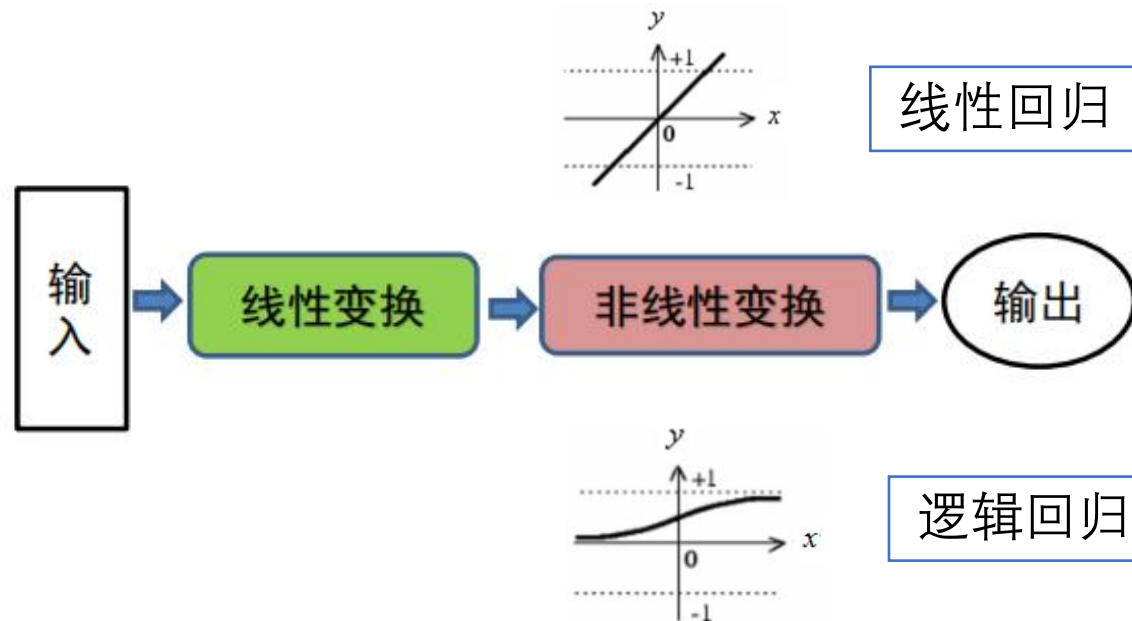
人工神经元 i 的模型图

$$y_i = f\left(\sum_{j=1}^n w_{ij}x_j + b_i\right) = f\left(\sum_{j=0}^n w_{ij}x_j\right) = f(\mathbf{w}^T \mathbf{x})$$

($x_0 = 1, w_{i0} = b_i$)

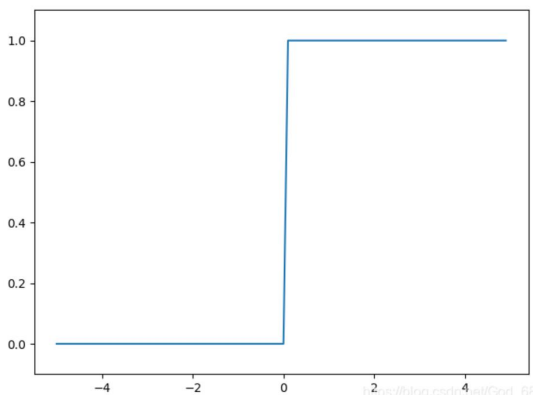
线性加权

非线性变换

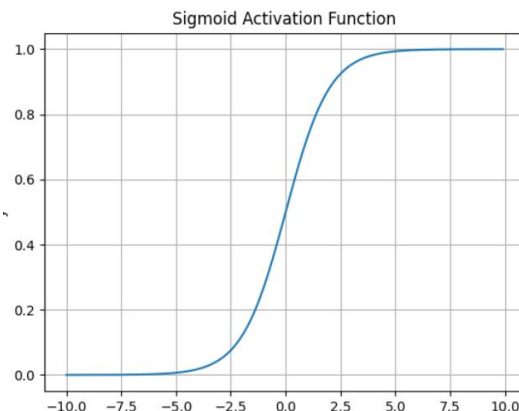


- 输入：一个向量
- 输出：一个标量
- 运算：
 - 线性变换 (加权求和)
 - 非线性变换 (非线性函数)

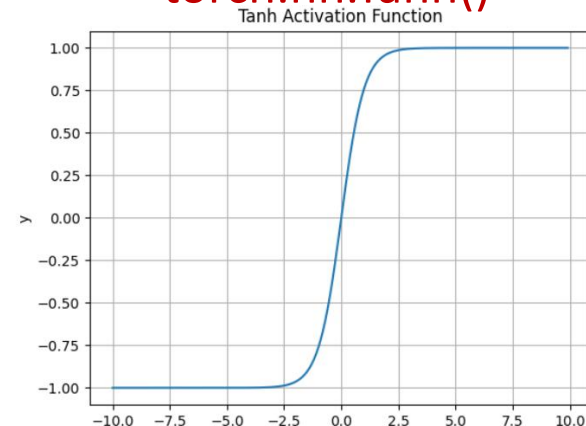
阶跃函数 $f(x) = \begin{cases} 1 & x = 0 \\ 0 & \text{others} \end{cases}$



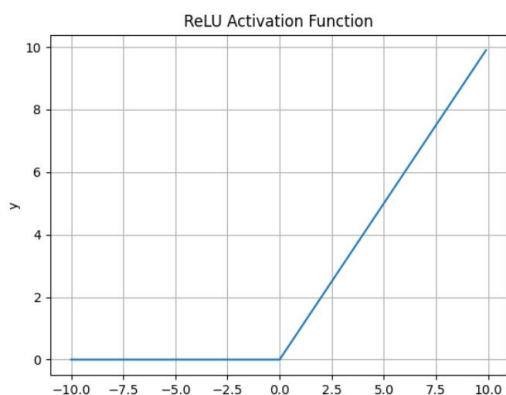
S型函数 $f(x) = \frac{1}{1 + e^{-x}}$
`torch.nn.Sigmoid()`



双曲正切函数 $f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
`torch.nn.Tanh()`

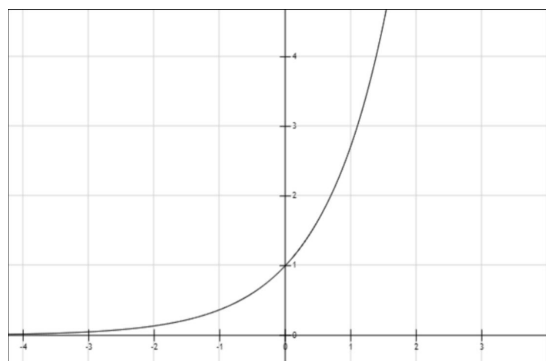


修正线性单元 $f(x) = \max(0, x)$



ReLU (Rectified Linear Unit)
`torch.nn.ReLU()`

$\text{Softmax}(x) = \frac{e^{x_i}}{\sum_i e^{x_i}}$

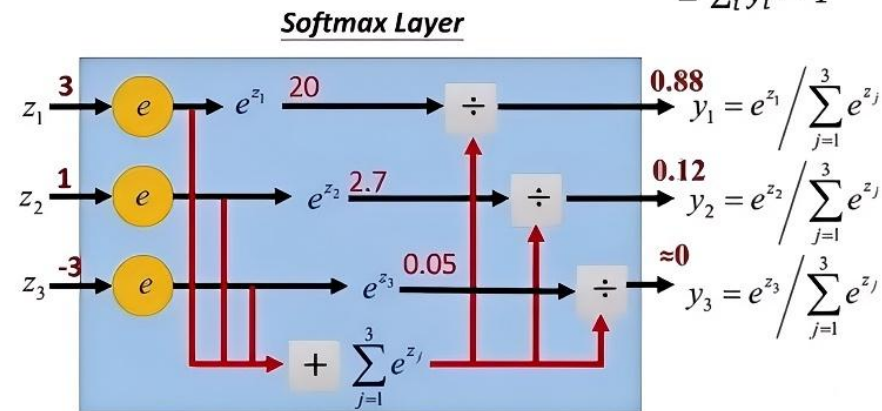


$y = e^x$ 函数

- Softmax layer as the output layer

Probability:

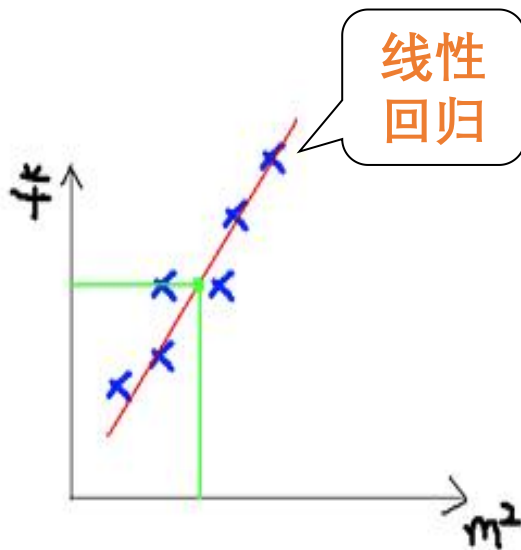
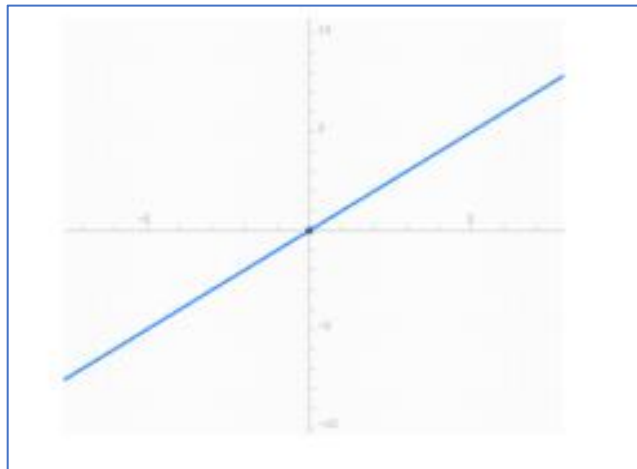
- $1 > y_i > 0$
- $\sum_i y_i = 1$



`torch.nn.Softmax()`

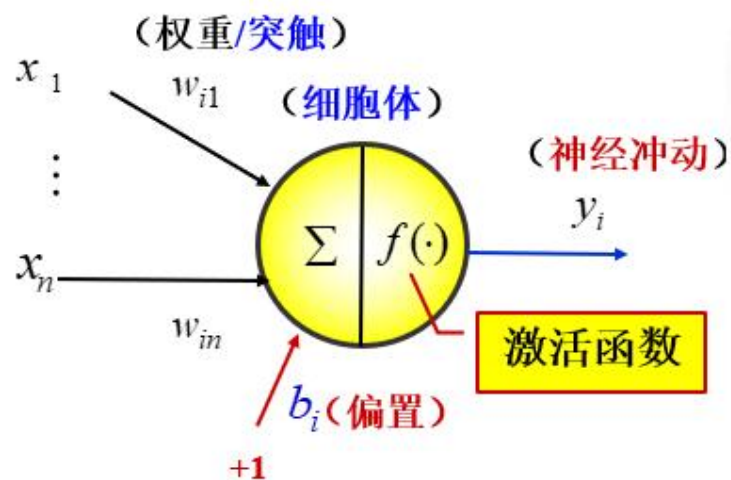
- linear函数
- 特点
 - 输入是什么，输出就是什么
 - 没有做任何 非线性变换
- 用法
 - 仅仅用于线性回归
- 缺点
 - 没有非线性变换
 - 无法拟合非线性函数
 - 无法建立非线性模型

$$y = f(x) = x$$



$$\text{房价} = \text{面积} * w + b$$

没有非线性变换的“激活函数”



• sigmoid函数也叫 Logistic 函数

• 特点:

- 它可以将一个实数映射到(0,1)的区间
- 在特征相差不是特别大时效果比较好

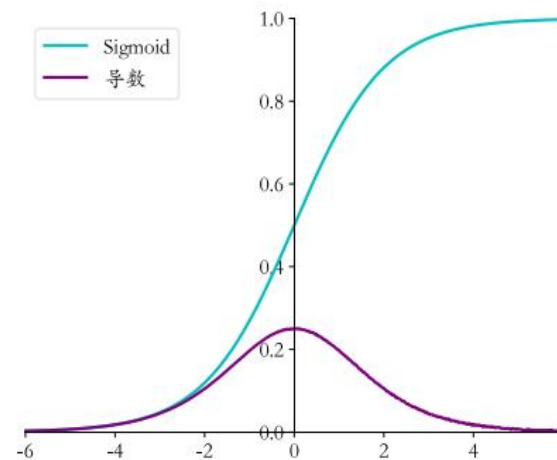
• 用法:

- 通常用来做二分类

• 缺点:

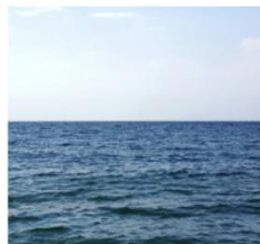
- 激活函数计算量大
- 容易出现梯度消失
 - 当数据分布在曲线平滑位置时很容易就会出现梯度消失

$$f(x) = \text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$



$$f'(x) = f(x)(1 - f(x))$$

梯度消失



$$0.25^8 \approx 2.3 \times 10^{-10}$$

早期最流行的激活函数，激活函数的代名词

假设现有一个MLP (d 层, 为简单而没有偏置参数)

$$f_t(\mathbf{h}^{t-1}) = \sigma(\mathbf{W}^t \mathbf{h}^{t-1}) \quad \sigma \text{ 是一个激活函数}$$

$$\frac{\partial \mathbf{h}^t}{\partial \mathbf{h}^{t-1}} = \text{diag}(\sigma'(\mathbf{W}^t \mathbf{h}^{t-1})) \mathbf{W}^t \quad \sigma' \text{ 是 } \sigma \text{ 的导函数}$$

$$\prod_{i=t}^{d-1} \frac{\partial \mathbf{h}^{i+1}}{\partial \mathbf{h}^i} = \prod_{i=t}^{d-1} \text{diag}(\sigma'(\mathbf{W}^i \mathbf{h}^{i-1})) \mathbf{W}^i$$

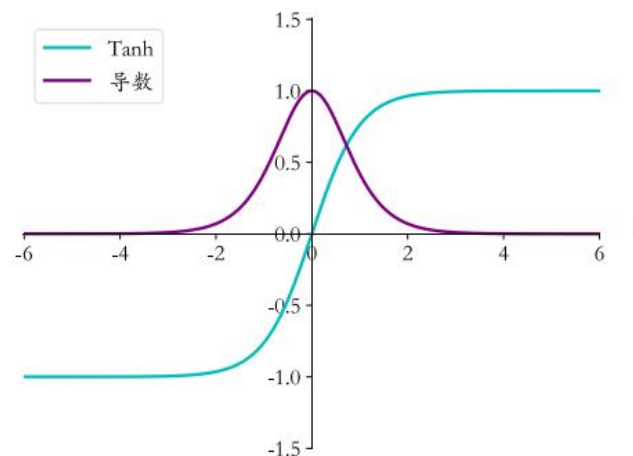
- **Tanh函数** 也称为双切正切函数
- **特点:**
 - 取值范围为 $[-1,1]$
 - 输出是以0为中心
 - 可以认为是一个放大的sigmoid函数
- **用法:**
 - 实际中tanh会比sigmoid更常用
 - 循环神经网络中常用
 - 靠近输出值位置
 - 二分类问题
- **缺点:**
 - 梯度消失
 - 在曲线几乎水平的区域学习非常的慢

- 1、解决了原点对称问题;
- 2、收敛速度比sigmoid快

$$f(x) = \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$\tanh(x) = 2\text{sigmoid}(2x) - 1$$

$$\tanh'(x) = 4\text{sigmoid}'(x)$$



Sigmoid函数的改进版

非零中心化的输出会使得其后的神经元的输入发生偏置偏移 (bias shift), 并进一步使得梯度下降的收敛速度变慢

ReLU函数p214

ReLU (rectified linear unit) 是Nair & Hinton是在2010为限制玻尔兹曼机 (restricted Boltzmann machines)提出的, 并且首次成功地应用于深度神经网络(Xavier Glorot, 2011)

- ReLU函数 (矫正的线性单元)

- Rectified Linear Unit(ReLU)

- 特点:

- 大于0的部分输出为数据本身
 - 小于0的部分输出为0
 - ReLU 对于梯度收敛有巨大加速作用
 - 只需要一个阈值就可以得到激活值节省计算量

- 用法:

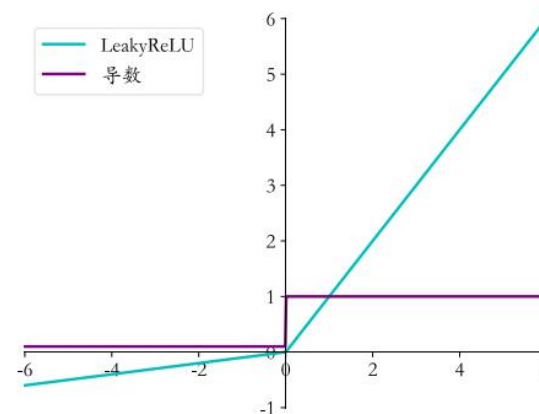
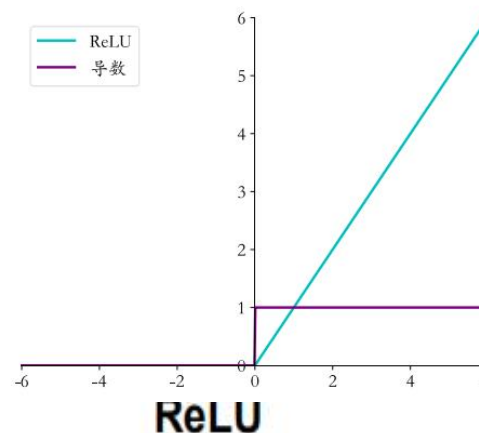
- 深层网络中隐藏层常用

- 缺点:

- 过于生猛, 一言不合就判4刑, 直接使数据变为0, 从此节点后相关信息全部丢失

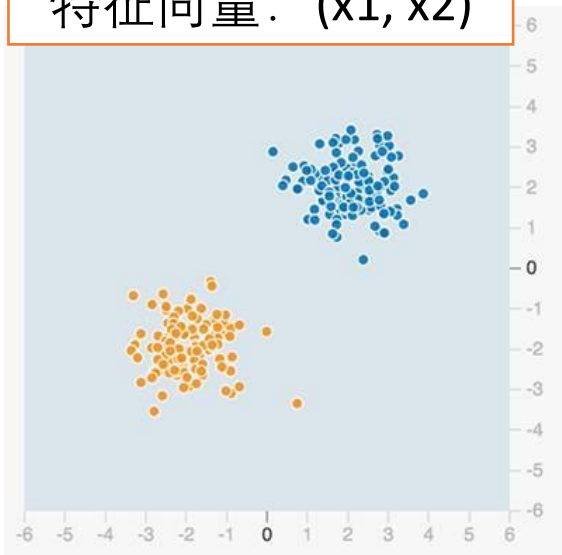
- 1、解决了部分梯度消失问题
 - 2、收敛速度更快

$$f(x) = \max(0, x) = \begin{cases} x & x > 0 \\ 0 & x \leq 0 \end{cases}$$

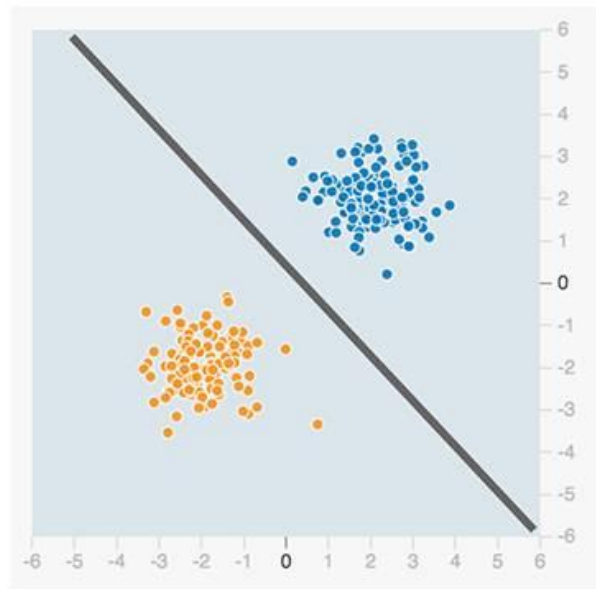


$y = \max(\alpha x, x)$ Leaky ReLU
 $\alpha = \text{small const. (e.g. 0.1)}$

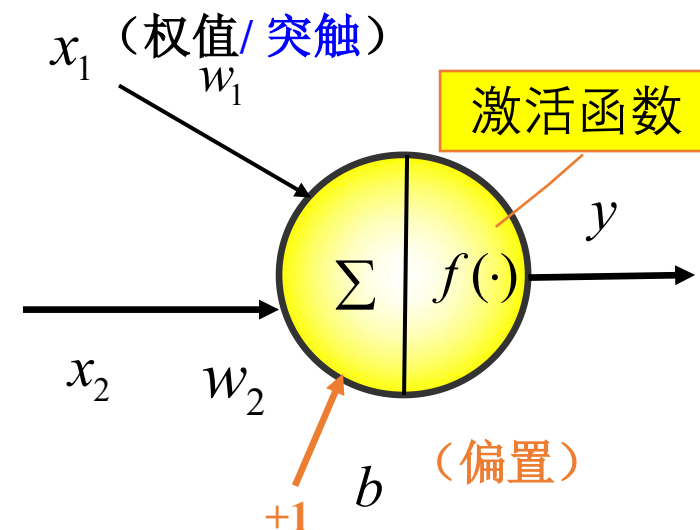
激活函数中的重要发明

特征向量: (x_1, x_2) 

每一个数据点 (x_1, x_2) ,
如何区分一个数据
点是橙色的还是蓝
色的?



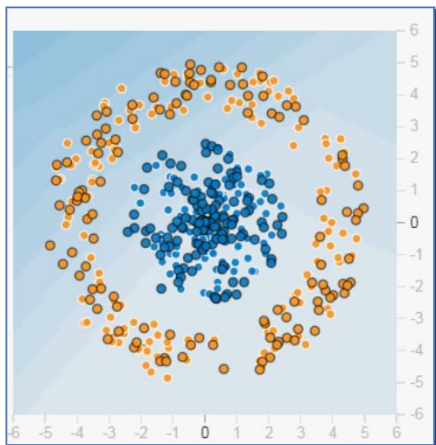
分界线: $w_1x_1 + w_2x_2 + b = 0$
蓝色点: $w_1x_1 + w_2x_2 + b > 0$
黄色点: $w_1x_1 + w_2x_2 + b < 0$



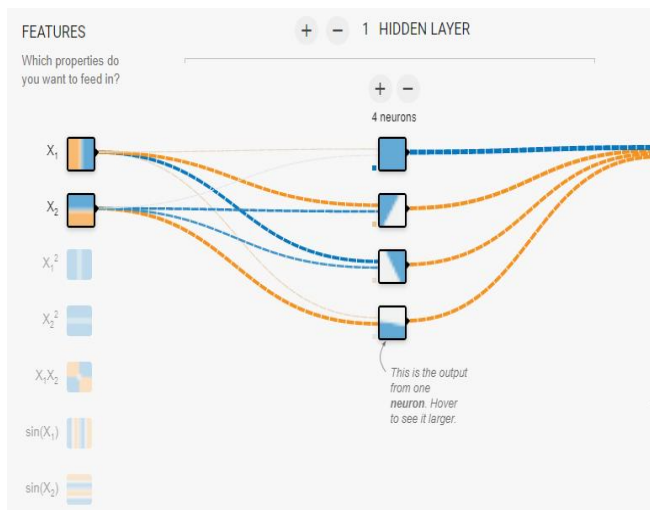
人工神经元的模型图

$$y = f(w_1x_1 + w_2x_2 + b)$$

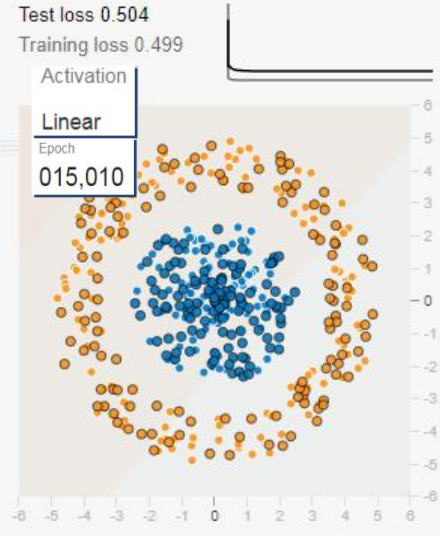
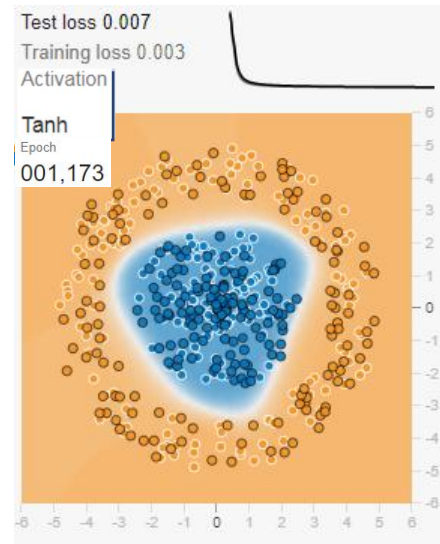
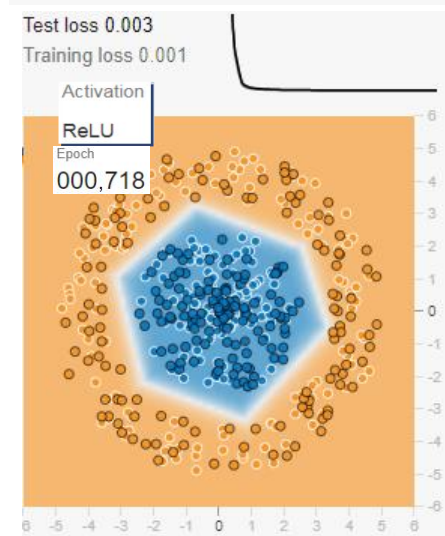
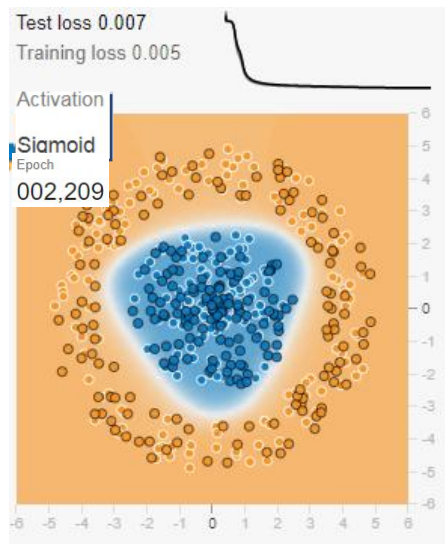
一个神经元实现左上图分类问题,
其 $f(\cdot)$ 可以采用 **Linear, Sigmoid, Tanh, ReLU** 作为激活函数, 并对神经
元输出做后处理。



非线性二分类问题



含有1个隐藏层的全连接神经网络



$$y_i = f\left(\sum_{j=1}^n w_{ij}x_j + b_i\right) = f\left(\sum_{j=0}^n w_{ij}x_j\right)$$

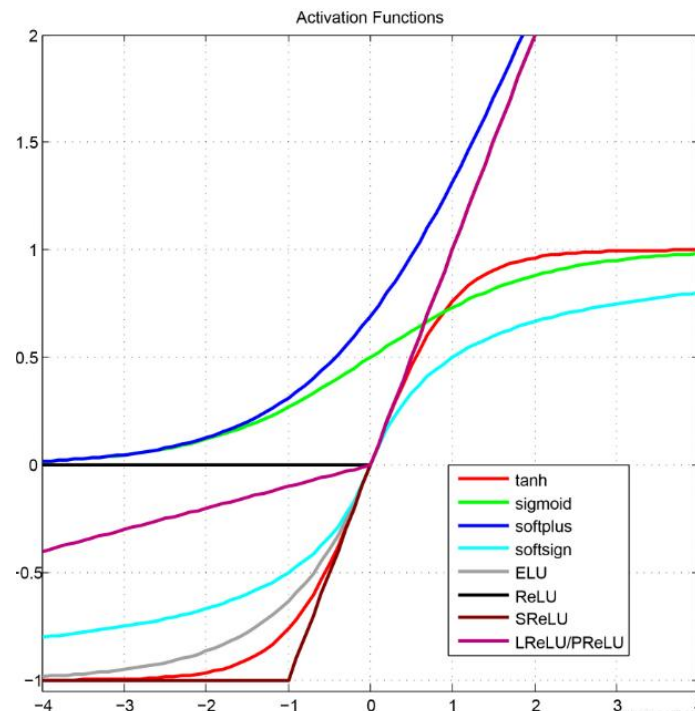
$$= f(\mathbf{w}^T \mathbf{x}) \quad (x_0 = 1, w_{i0} = b_i)$$

线性加权

非线性激活函数

如果不用非线性激活函数，每一层都是上一层的线性函数，无论神经网络多少层，输出都是输入的线性组合。

Sigmoid	$S(x) = 1/(1 + e^{-x})$	
tanh	$\tanh(x)$	
ReLU	$\max(0, x)$	Nair et al., ICML 2010
Leaky ReLU	$\max(ax, x)$, a is fixed	Maas et al., ICML2013
PReLU	$\max(ax, x)$, a is learned	He et al., ICCV2015
RReLU	$\max(ax, x)$, a is a random variable	Xu et al., 2015
maxout	$\max(w_1x + b_1, w_2x + b_2, \dots)$	Goodfellow et al., ICML2013
ELU	$\begin{cases} x & x > 0 \\ a(e^x - 1) & x \leq 0 \end{cases}$	Clevert et al., 2015
CReLU	$(\max(x, 0), \max(-x, 0))$	Shang et al., 2016



思考：激活函数的作用以及应用中如何选择合适的激活函数？

高斯误差线性单元 (Gaussian Error Linear Units, [GeLU](#), 2016) 在Transformer 模型 (谷歌的 BERT 和 OpenAI 的 GPT-2) 中得到了应用。

SwiGLU & **GeGLU** (GLU, [Gated Linear Units](#)) : LLaMA、ChatGLM。

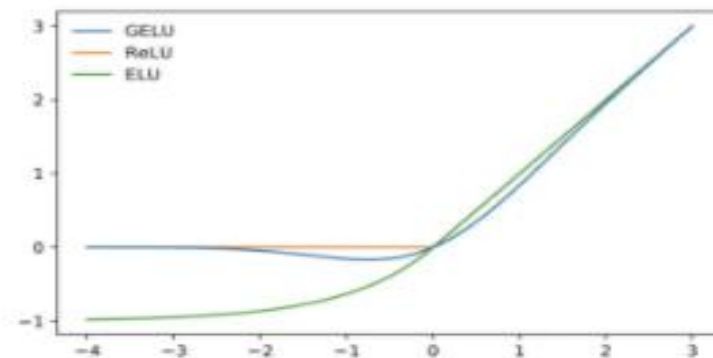


Figure 1: The GELU ($\mu = 0, \sigma = 1$), ReLU, and ELU ($\alpha = 1$).



Model	Category	Size	Normalization	PE	Activation	Bias	#L	#H	d_{model}	MCL
GPT3 [55]	Casual decoder	175B	Pre Layer Norm	Learned	GeLU	✓	96	96	12288	2048
PanGU- α [73]	Casual decoder	207B	Pre Layer Norm	Learned	GeLU	✓	64	128	16384	1024
OPT [79]	Casual decoder	175B	Pre Layer Norm	Learned	ReLU	✓	96	96	12288	2048
PaLM [56]	Casual decoder	540B	Pre Layer Norm	RoPE	SwiGLU	×	118	48	18432	2048
BLOOM [66]	Casual decoder	176B	Pre Layer Norm	ALiBi	GeLU	✓	70	112	14336	2048
MT-NLG [90]	Casual decoder	530B	-	-	-	-	105	128	20480	2048
Gopher [59]	Casual decoder	280B	Pre RMS Norm	Relative	-	-	80	128	16384	-
Chinchilla [34]	Casual decoder	70B	Pre RMS Norm	Relative	-	-	80	64	8192	-
Galactica [35]	Casual decoder	120B	Pre Layer Norm	Learned	GeLU	×	96	80	10240	2048
LaMDA [85]	Casual decoder	137B	-	Relative	GeGLU	-	64	128	8192	-
Jurassic-1 [89]	Casual decoder	178B	Pre Layer Norm	Learned	GeLU	✓	76	96	13824	2048
LLaMA [57]	Casual decoder	65B	Pre RMS Norm	RoPE	SwiGLU	✓	80	64	8192	2048
GLM-130B [80]	Prefix decoder	130B	Post Deep Norm	RoPE	GeGLU	✓	70	128	10240	2048
T5 [71]	Encoder-decoder	11B	Pre RMS Norm	Relative	ReLU	×	24	128	1024	-

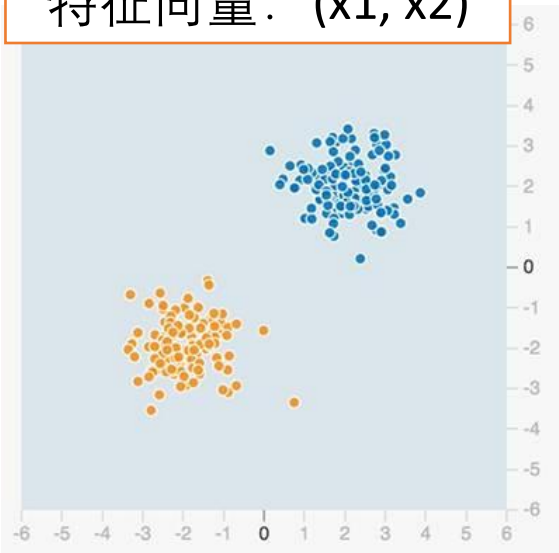
GeLU/GELU的数学形式简化计算为 $0.5x(1 + \tanh[\sqrt{2/\pi}(x + 0.044715x^3)])$

SwiGLU和GeGLU为 $\text{GEGLU}(x, W, V, b, c) = \text{GELU}(xW + b) \otimes (xV + c)$

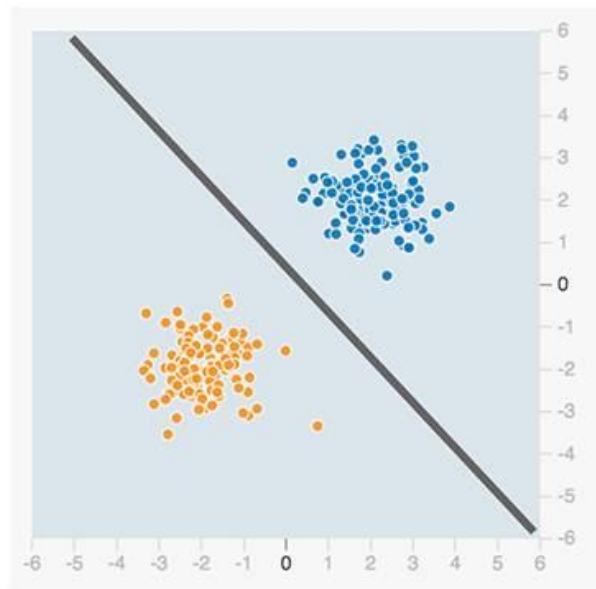
$\text{SwiGLU}(x, W, V, b, c, \beta) = \text{Swish}_\beta(xW + b) \otimes (xV + c)$

这里 $\text{Swish}_\beta(x) = x\sigma(\beta x)$, β 为指定常数, 如1

特征向量: (x_1, x_2)

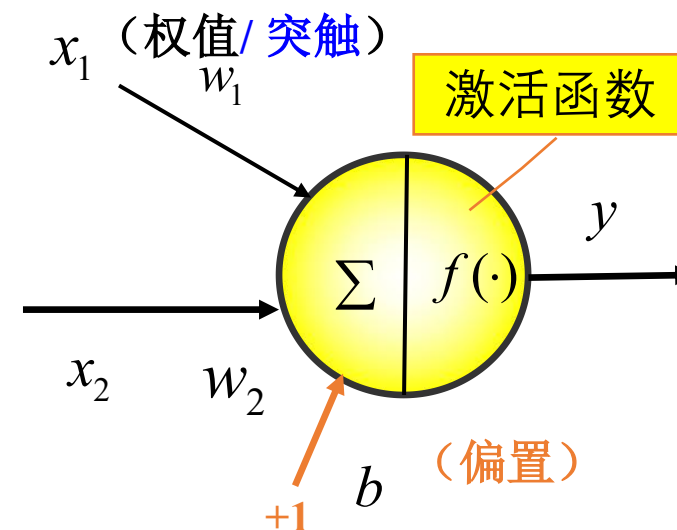


每一个数据点 (x_1, x_2) ,
如何区分一个数据
点是橙色的还是蓝
色的?



分界线: $w_1x_1 + w_2x_2 + b = 0$
蓝色点: $w_1x_1 + w_2x_2 + b > 0$
黄色点: $w_1x_1 + w_2x_2 + b < 0$

问题的关键: 必须为 w_1 、 w_2 和 b (可作为 w_0)
找到合适的值——即所谓的参数值, 然后
指示计算机如何分类这些数据点



人工神经元的模型图

$$y = f(w_1x_1 + w_2x_2 + b)$$

一个神经元实现左上图分类问题,
其 $f(\cdot)$ 可以采用 Linear, Sigmoid,
Tanh, ReLU 作为分类函数, 并对神经
元输出做后处理。

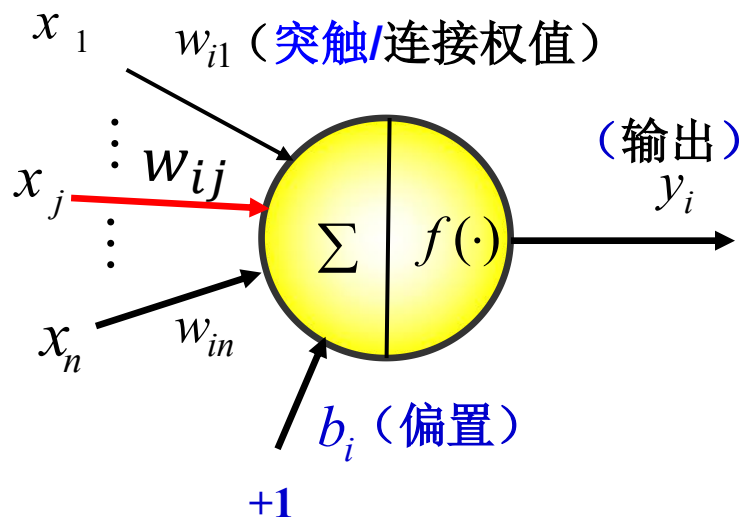
- 单神经元的学习：调整单神经元的连接权值，使输入输出具有需要的特性。

学习率

- 单神经元的连接权修正公式： $w_{ij}(k+1) = w_{ij}(k) + \alpha z_{ij}(k)$

学习信号

(输入)


 人工神经元 i 的模型图

$$y_i = f\left(\sum_{j=1}^n w_{ij} x_j + b_i\right) = f\left(\sum_{j=0}^n w_{ij} x_j\right) = f(\mathbf{w}^T \mathbf{x})$$

$$(x_0 = 1, w_{i0} = b_i)$$



唐纳德·赫布

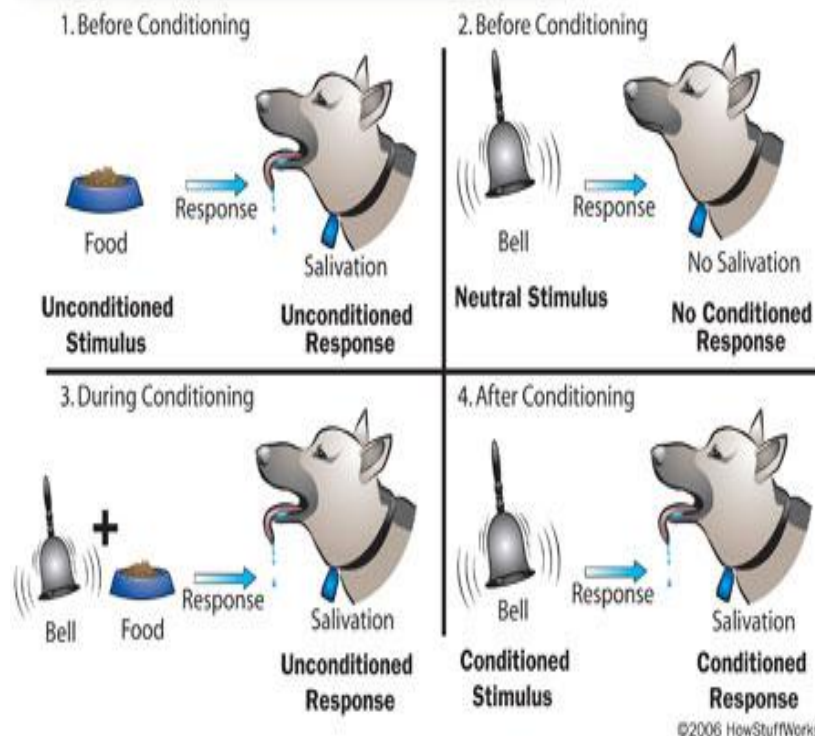
- 某一突触两端的神经元同步激活时，该连接的强度增强，反之减弱。

$$\Delta w_{ij}(k) = \alpha y_j(k) y_i(k), \quad \alpha > 0$$

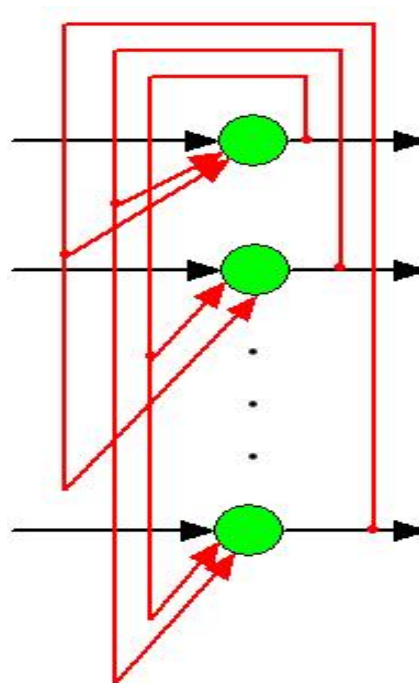
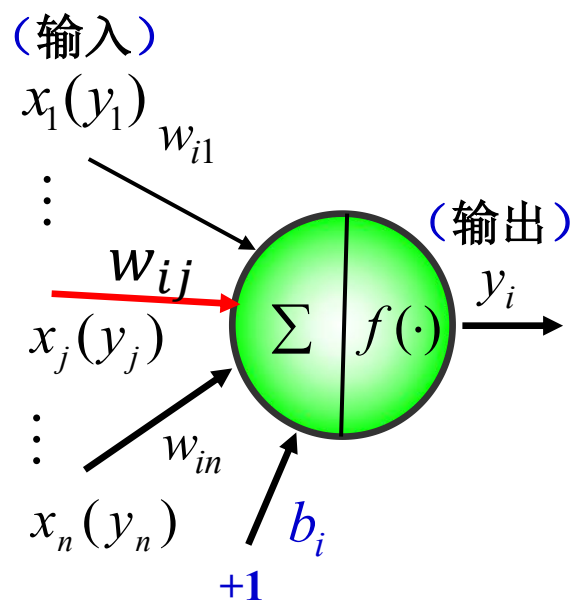
$$w_{ij}(k+1) = w_{ij}(k) + \Delta w_{ij}(k)$$

学习率

How Dog Training Works



巴甫洛夫的条件反射实验



Hopfield神经网络

1960年Widrow提出Delta学习规则，被应用到感知机（1957，Rosenblatt）。

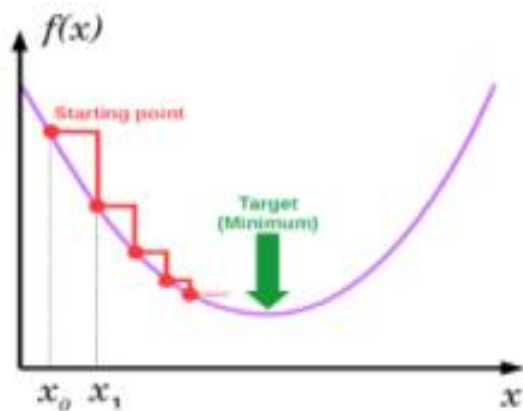
$$w_{ij}(k+1) = w_{ij}(k) + \Delta w_{ij}(k)$$

$$\Delta w_{ij}(k) = \alpha x_j(k) e_i(k), \quad \alpha > 0$$

$$e_i(k) = d_i - y_i(k)$$

期望
输出

学习率

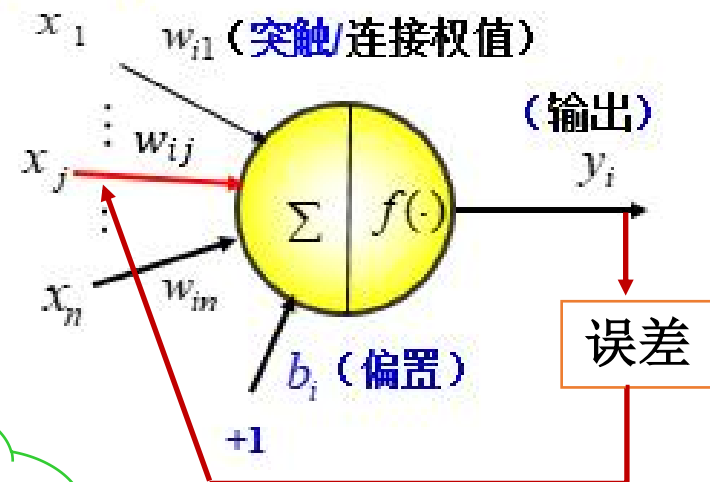


思考：如果激活函数 f 不同、目标函数 J 不同，学习信号会如何？

$$\min J = \frac{1}{2} \sum_i [y_i(k) - d_i]^2, \quad y_i(k) = f(x(k)) = \sum_j x_j(k) w_{ij}(k) + b_i(k)$$

$$w_{ij}(k+1) = w_{ij}(k) - \alpha \frac{\partial J}{\partial w_{ij}(k)} = w_{ij}(k) - \alpha \frac{\partial J}{\partial y_i(k)} \frac{\partial y_i(k)}{\partial w_{ij}(k)}$$

(输入)



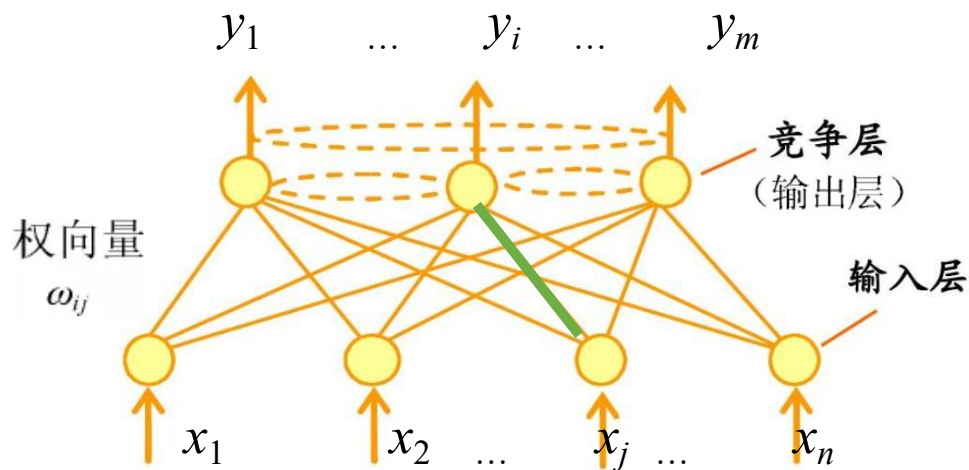

 Teuvo kohonen
(芬兰)

- 以某种内部规则（与外部环境无关）确定竞争层**获胜神经元**，对获胜神经元与输入间的连接权值进行调整，其余不变。

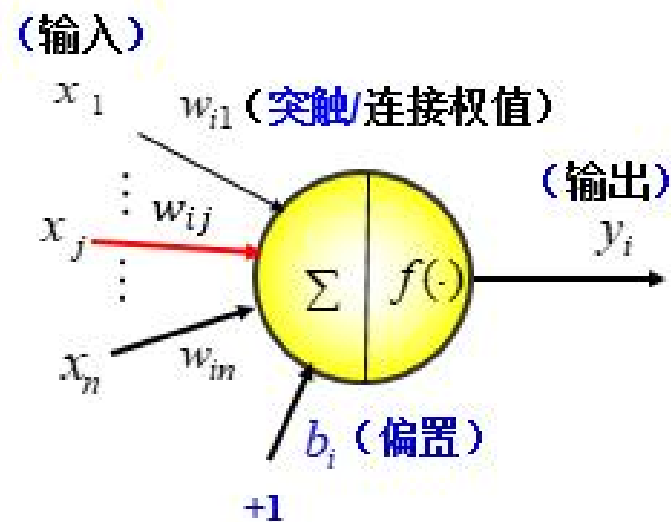
$$\Delta w_{ij}(k) = \alpha [x_j(k) - w_{ij}(k)], \quad \alpha > 0$$

$$w_{ij}(k+1) = w_{ij}(k) + \Delta w_{ij}(k)$$

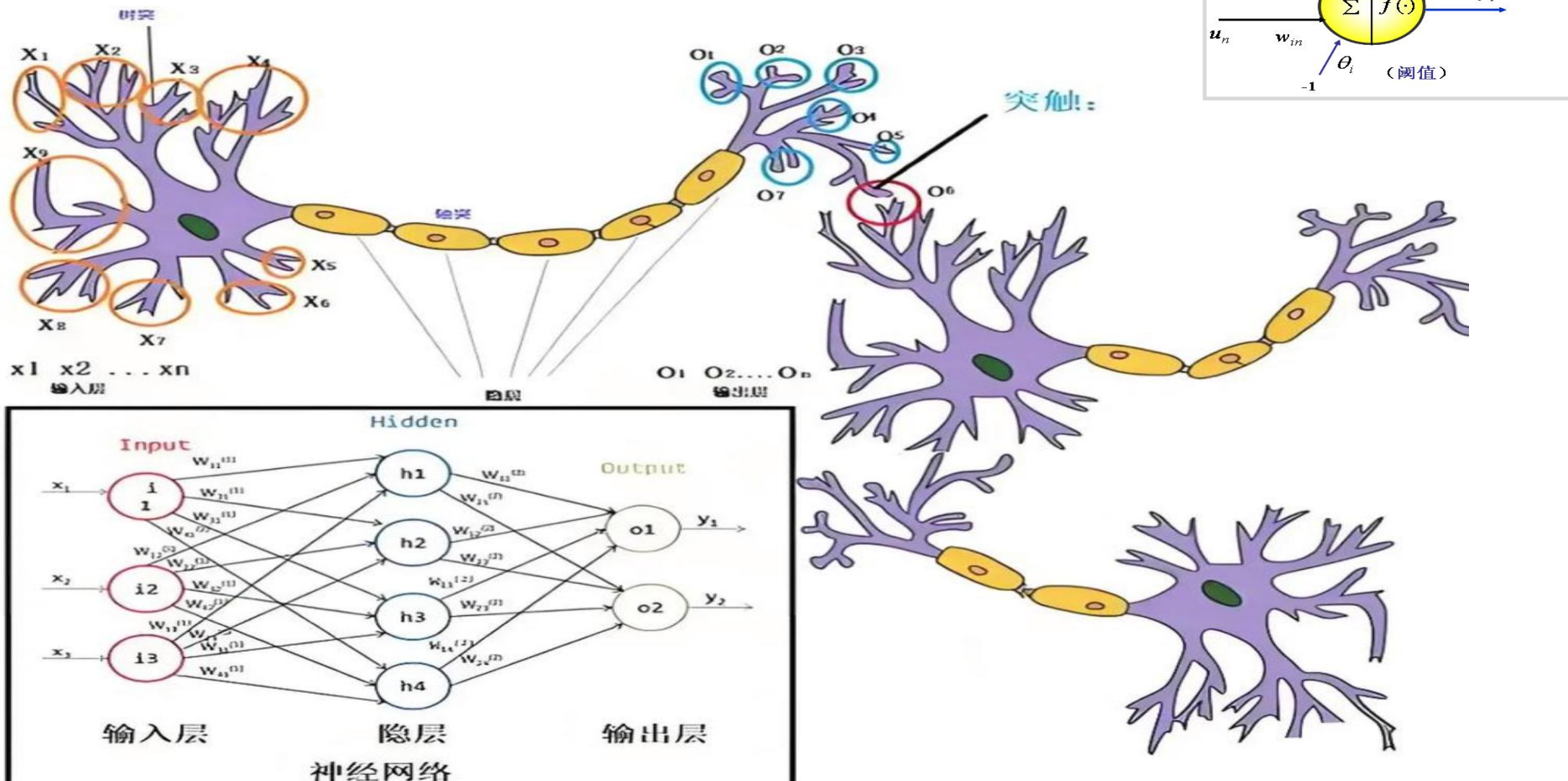
学习率



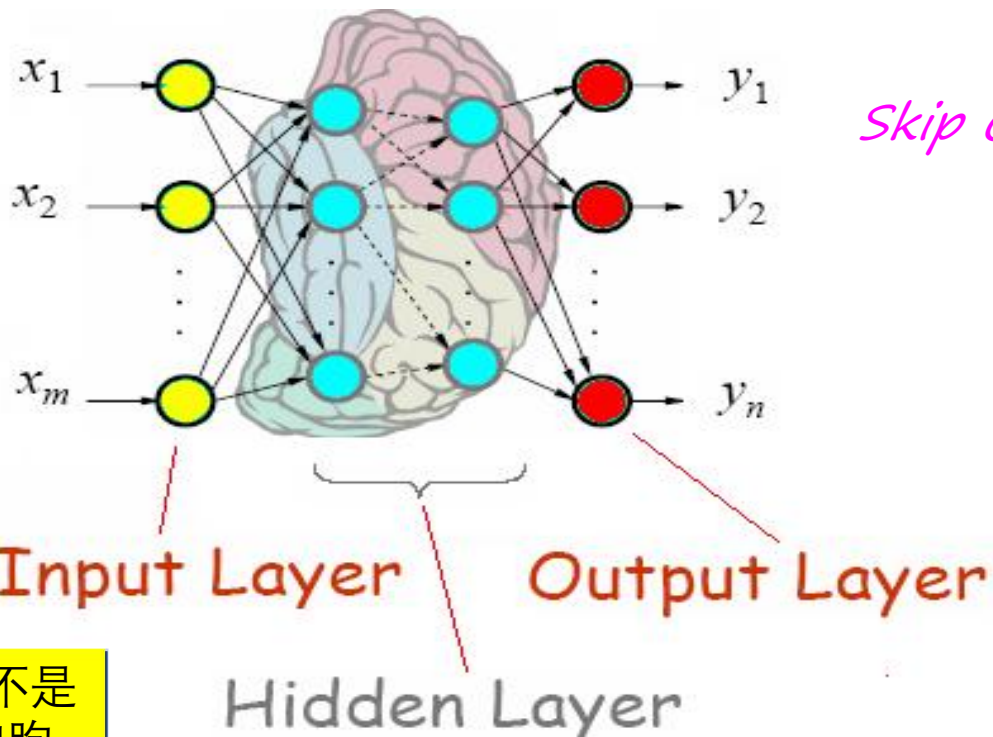
自组织神经网络的典型结构



人工神经网络是许多神经元的联合



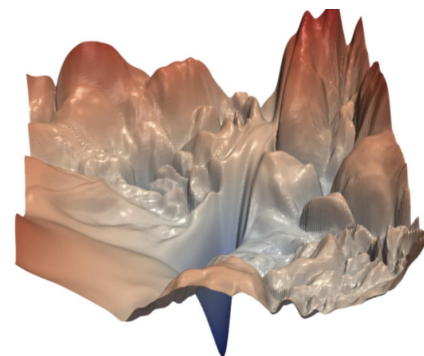
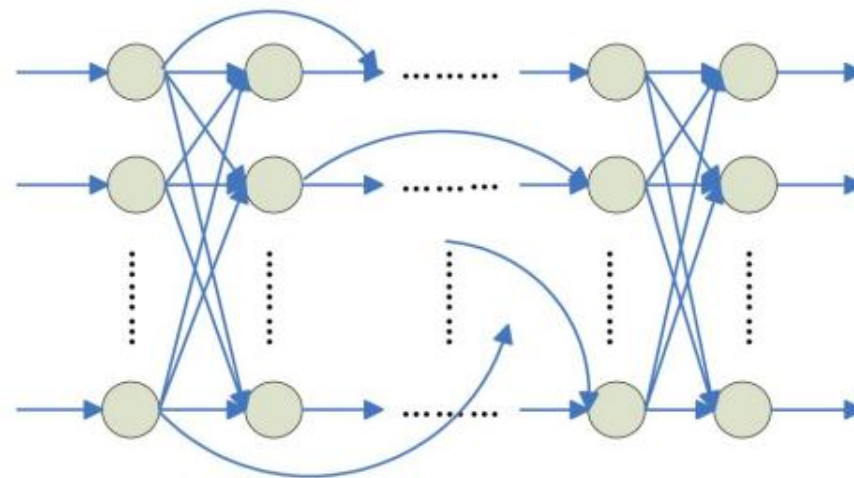
(1) 前馈型（前向型）

BP
神经网络输入层不是
神经细胞

- 全连接：每个神经元都和下一层的所有神经元相连

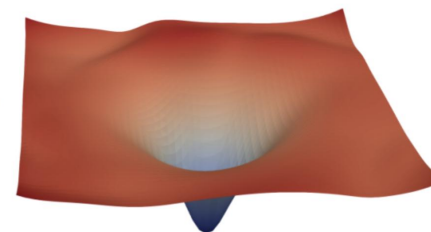
卷积神经网络（CNN）
也是前馈型的，但是局
部连接的

Skip Connections (或 *Shortcut Connections*)，跳跃连接



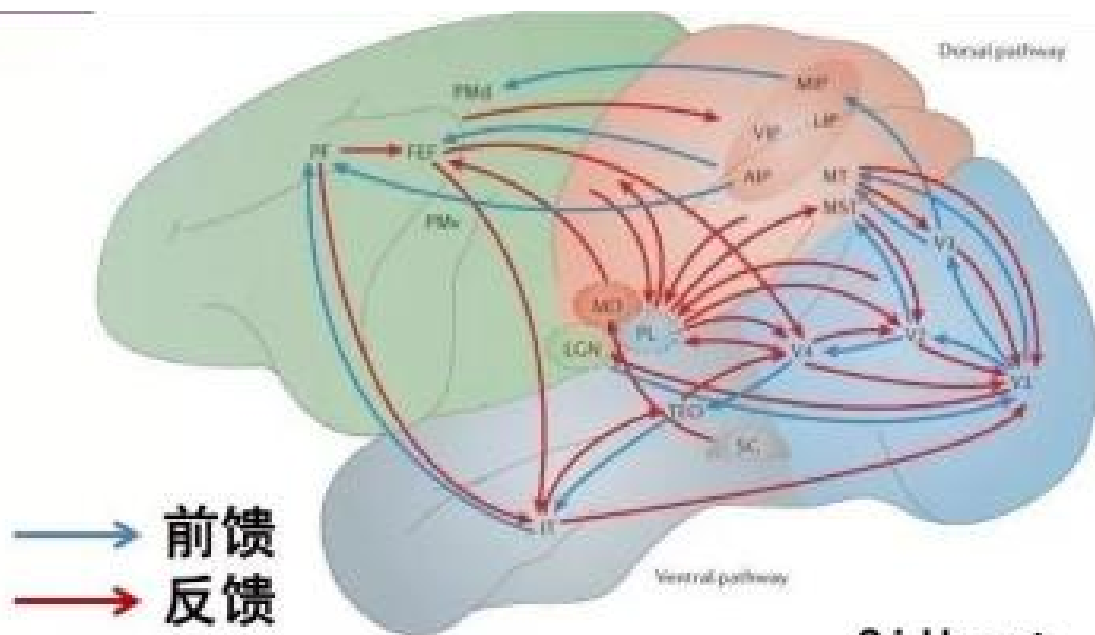
(a) without skip connections

ResNet-56 的损失表面



(b) with skip connections

(2) 反馈型

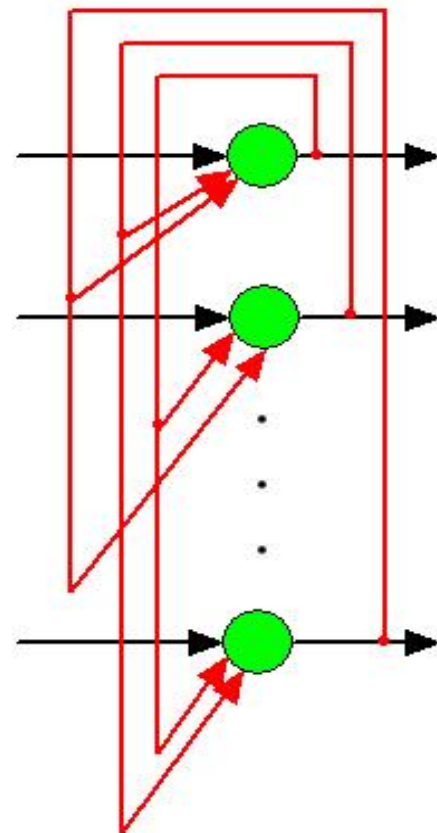
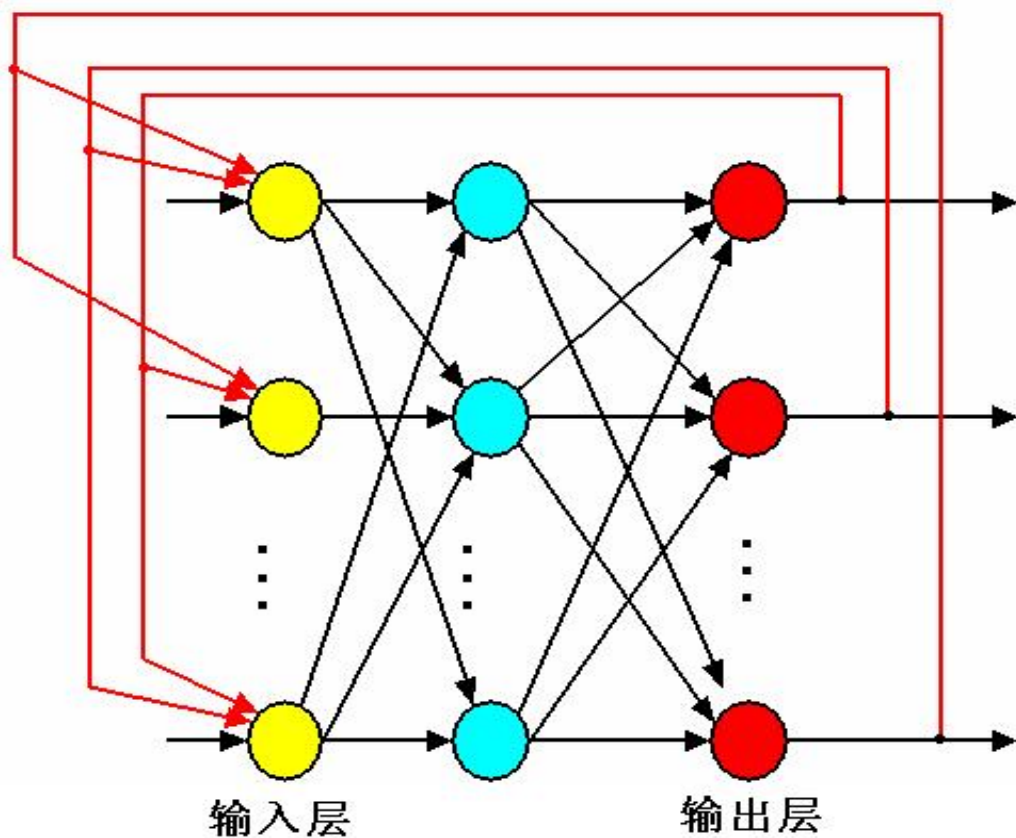


反馈连接在各级视觉皮层中广泛存在。例如初级视皮层 (V1) 4C 层神经元只有不超过8%的突触输入是来自于 LGN 前馈输入 (Peters et al., 1994)

Gilbert, Li,
2013

- 高级皮层的反馈对低级皮层的反应进行调制
- 同级皮层中的反馈也对神经元的反应进行调制
- 现在的用于图像处理的深度学习模型主要前馈模型，只是视觉皮层的一个粗糙近似

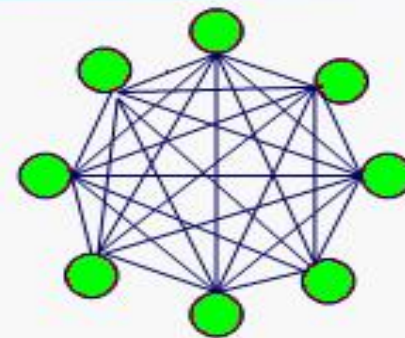
(2) 反馈型



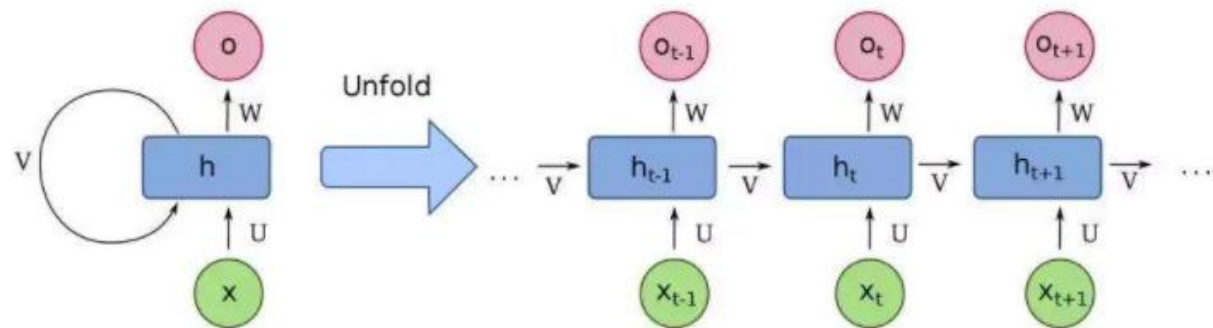
Hopfield神经网络

循环神经网络
(RNN, LSTM)
也是反馈型的

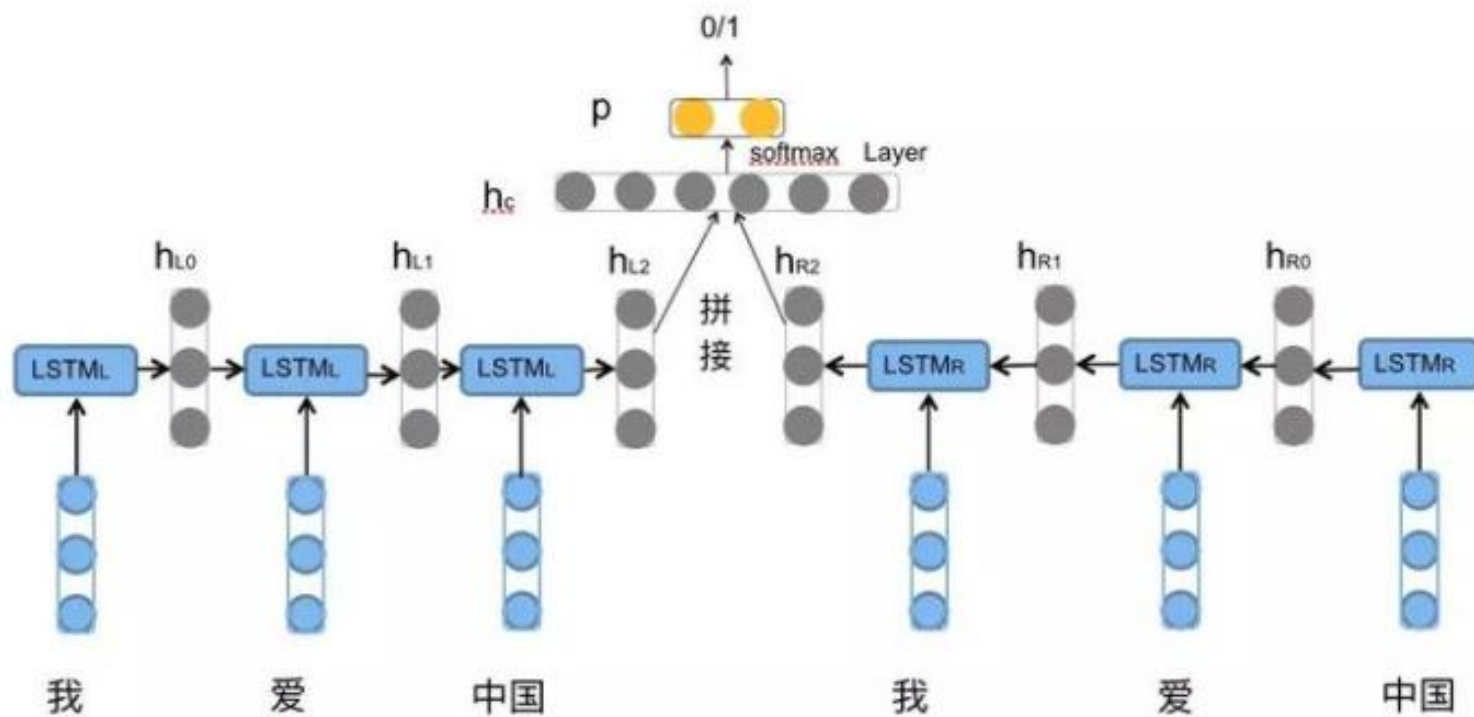
Hopfield Network



(2) 反馈型



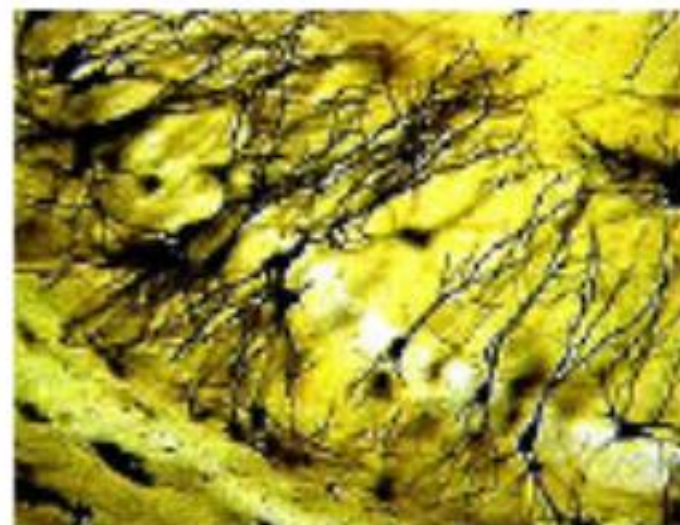
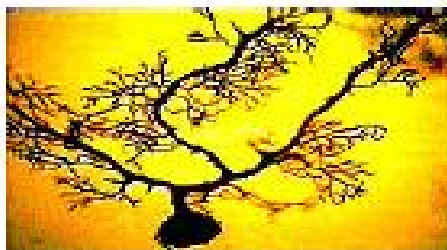
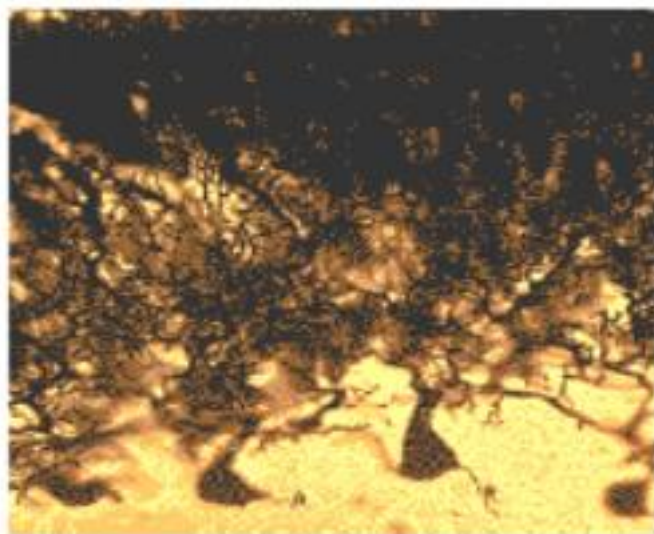
RNN模型



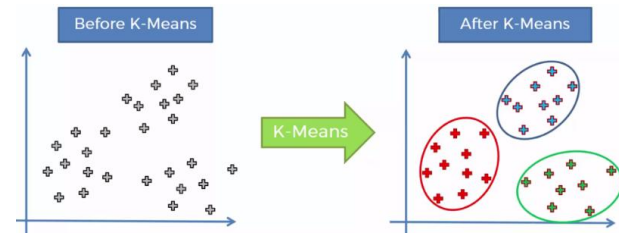
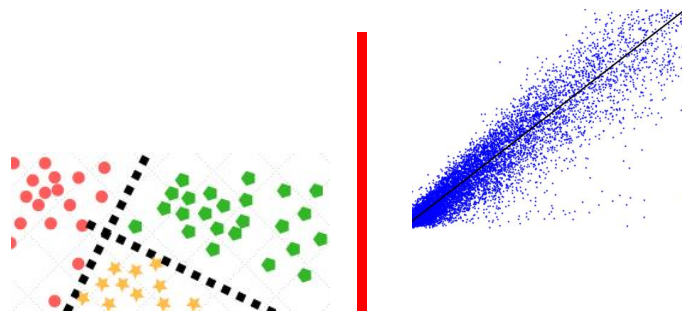
基于双向LSTM（长短期记忆网络）的情感分类

- 决定人工神经网络性能的3大要素：

- 神经元的特性——非线性激活函数；
- 神经元之间相互连接的形式——拓扑结构；
- 为适应环境而改善性能的学习规则。

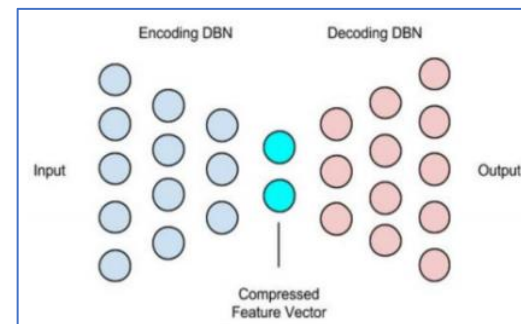
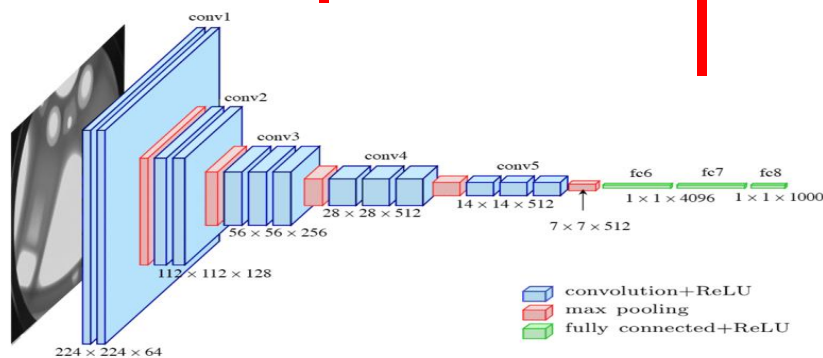


- ◆ 有监督学习
- ◆ 无监督学习
- ◆ 半监督学习

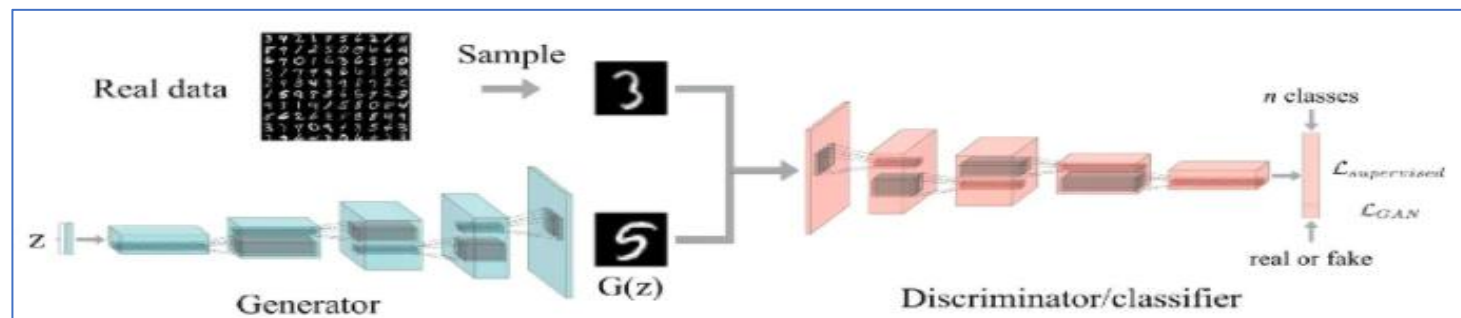


无监督学习：自动编码器、生成对抗网络**GAN**等

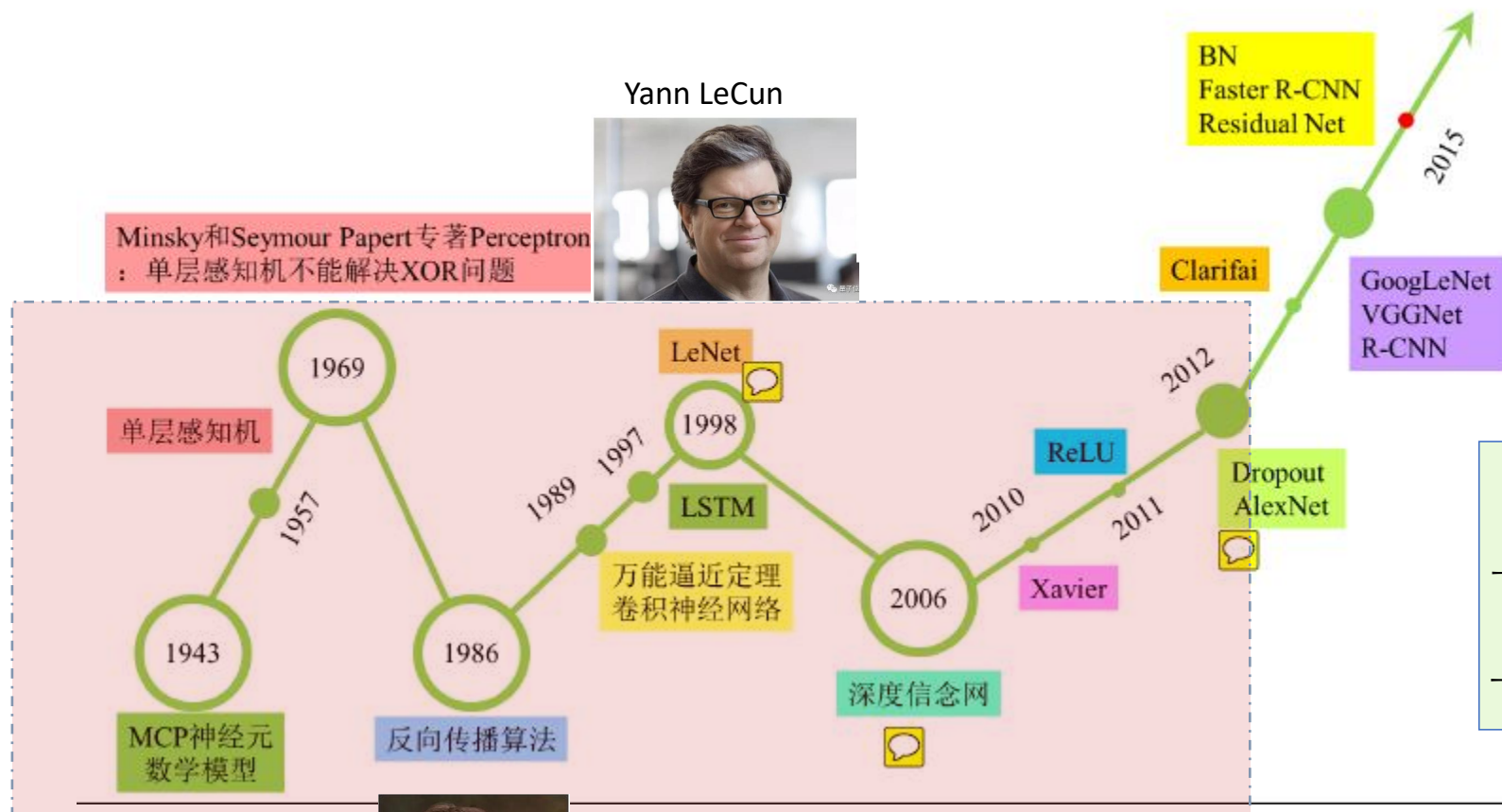
有监督学习：感知器、BP神经网络、卷积神经网络、循环神经网络RNN、LSTM等



半监督学习：生成对抗网络**GAN**、图卷积网络GCN等



本图来自于
“深度学习大
讲堂”微信公
众号



为什么深度学习在基础理论早就准备好的情况下，踟蹰不前了几十年，直到2012年之后才迎来一波爆发式的迅猛发展？



Geoffrey Hinton

2012年，Hinton课题组为了证明深度学习的潜力，首次参加ImageNet图像识别比赛，其通过构建的CNN网络AlexNet一举夺得冠军，且碾压第二名（SVM方法）的分类性能。也正是由于该比赛，CNN吸引到了众多研究者的注意。



Geoffrey Hinton, Yann LeCun, 和Yoshua Bengio共同获得了 2018年的图灵奖。



Geoffrey Hinton
加拿大多伦多大学

Yann LeCun
纽约大学

Yoshua Bengio
加拿大蒙特利尔大学

2019年3月27日 ——ACM宣布，深度学习的三位创造者Yoshua Bengio, Yann LeCun, 以及
Geoffrey Hinton获得了 2018年的图灵奖。



约翰·J·霍普菲尔德
(John J. Hopfield)

杰弗里·E·辛顿
(Geoffrey E. Hinton)

瑞典皇家科学院当地时间10月8日宣布，将2024年诺贝尔物理学奖授予约翰·J·霍普菲尔德（John J. Hopfield）和杰弗里·E·辛顿（Geoffrey E. Hinton），表彰他们在使用人工神经网络进行机器学习的基础性发现和发明。



- 神经网络基础：单层感知器、线性神经网络（ADALINE）、**BP神经网络**、Hopfield神经网络等
- 神经网络进阶：玻尔兹曼机、受限玻尔兹曼机、**循环神经网络**等
- 深度学习网络：深度置信网络、**卷积神经网络**、深度残差网络、LSTM网络、胶囊网络、生成对抗网络等
- 深度网络应用：应用于传统的数据挖掘与机器学习问题，**手写体识别**，**图像识别**，应用于**自然语言处理**，**AlphaGo**等



- 1. 有哪些深度学习框架以及部署推理框架？在实际应用中如何选择？
- 2. 人工神经元模型中有哪两个运算？
- 3. 激活函数的作用是什么，在应用中如何选择合适的激活函数？
- 4. 单个神经元的学习目的是什么？如何进行学习？主要的学习规则有哪些？在误差纠正学习规则中，若采用梯度下降法，学习信号是什么？



- 1.在<http://playground.tensorflow.org/>上练习螺旋形数据集的分类，给出你所设计的能实现正确分类的神经网络隐层的层数以及每层神经元的个数，并对比分析不同激活函数的特点。
- 2.课外查阅课上所提到的一些神经网络模型（例如卷积神经网络、胶囊网络、循环神经网络RNN、LSTM、自动编码器、生成对抗网络GAN、图卷积网络GCN、Transformer等），并选择你所感兴趣的模型（至少1个），用一句话介绍。